

Convolutional NNs

Reshad Hosseini

Department of Machine
Intelligence and
Robotics

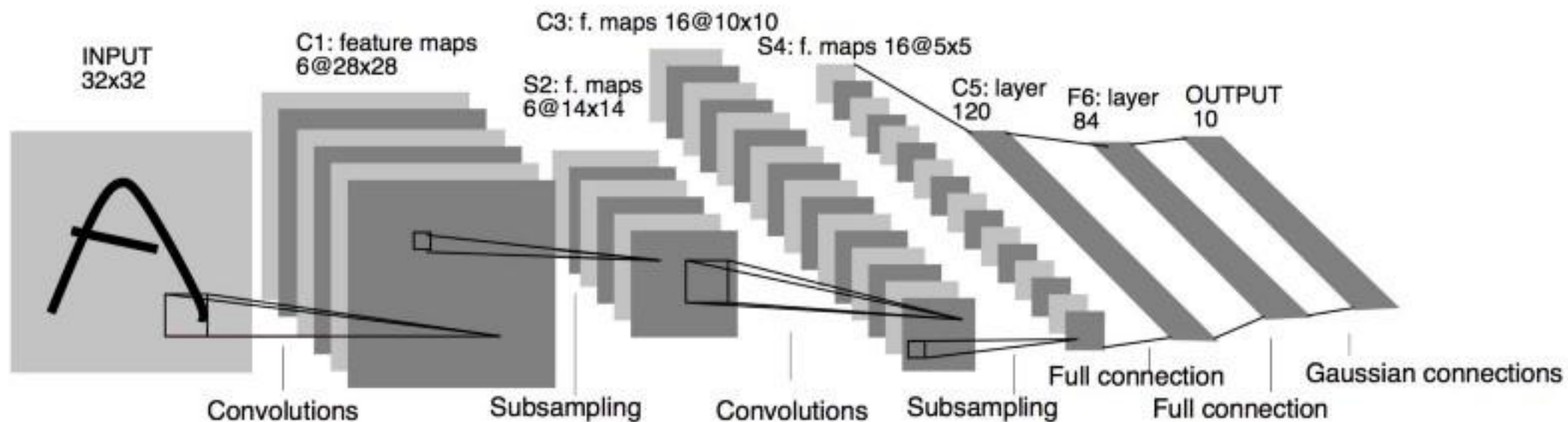
University of Tehran

Hamrah Academy Workshop





Motivation



History: gradient-based learning applied to document recognition [LeCun, Bottou, Bengio, Haffner 1998]



Motivation: Classification & Retrieval

Classification



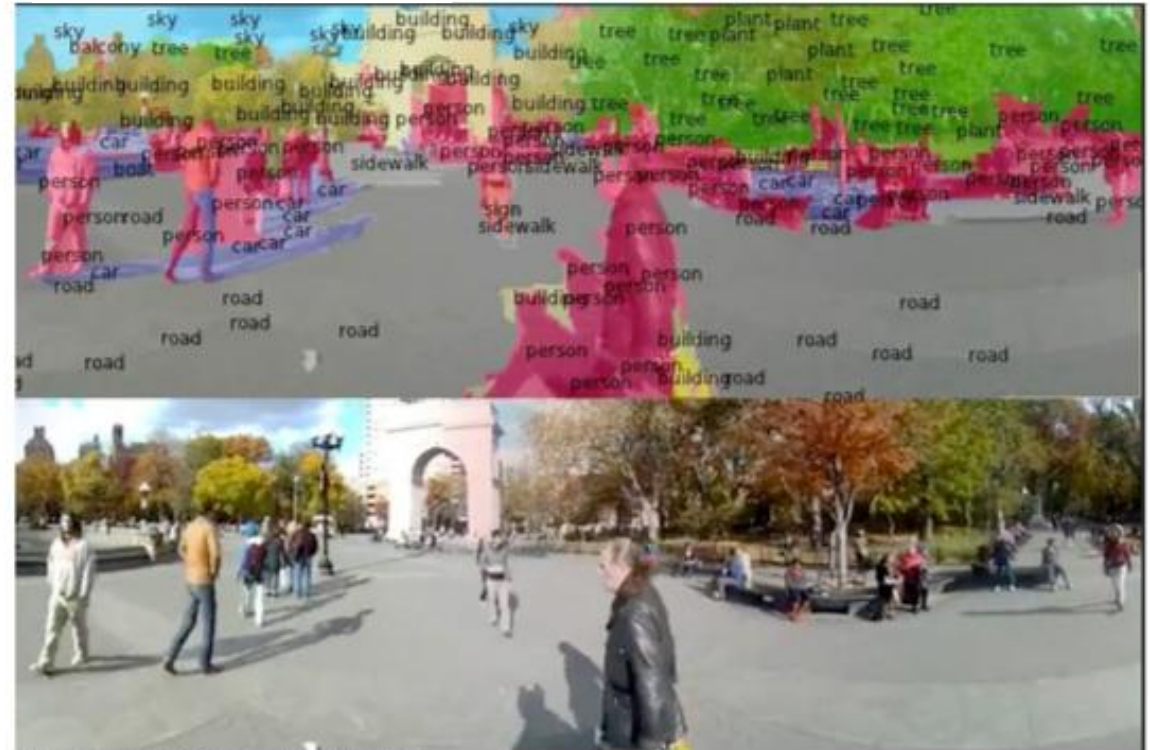
Retrieval



[Krizhevsky et al., 2012]



Segmentation

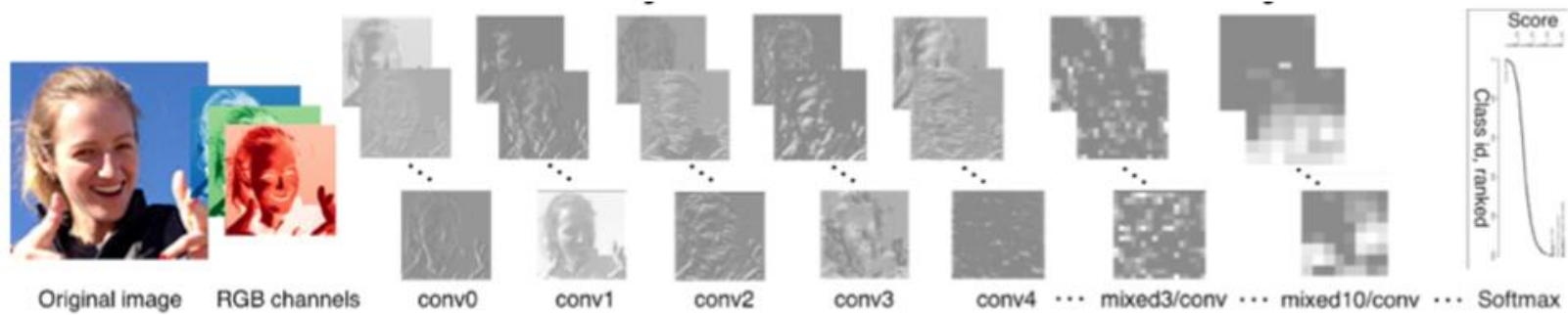


[Farabet et al., 2012]

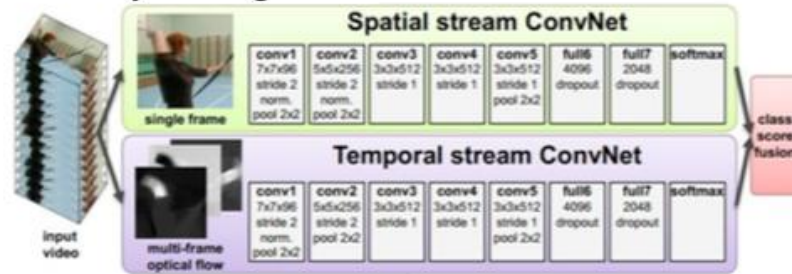


Motivation: Verification & Activity Recognition

Face Verification

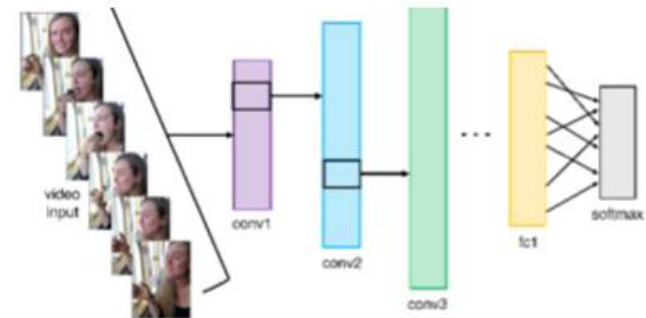


Activity Recognition



[Simonyan et al., 2014]

[Taigman et al., 2014]





Motivation: Pose Estimation and RL

Pose Estimation



[Yoshev and Szegedy, 2014]

Real-time Atari Game

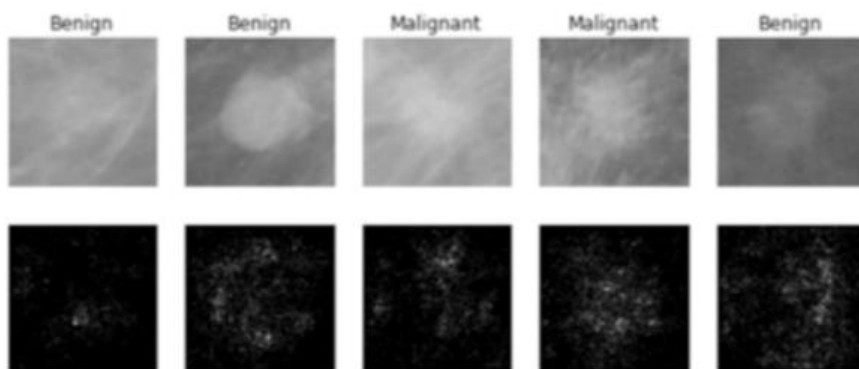


[Guo et al., 2014]



Motivation: Classification in ...

Breast Mass Classification



[Levy et al., 2016]

Galaxy Morphology Classification



[Dieleman et al., 2014]

Traffic Sign Classification



[Ciresan et al., 2011]

[Sermanet et al., 2011]



Motivation

Killer Whale Classification



[Bergler et al., 2011]

Road Detection



[Mnih and Hinton, 2010]



Motivation: Image Captioning

No errors



A white teddy bear sitting in the grass



A man riding a wave on top of a surfboard

Minor errors



A man in a baseball uniform throwing a ball



A cat sitting on a suitcase on the floor

Somewhat related



A skateboarder does a trick on a ramp



A woman standing on a beach holding a surfboard

Image Captioning

[Vinyals et al., 2015]
[Karpathy and Fei-Fei, 2015]



CNNs in 2012: "SuperVision" (aka "AlexNet")

"AlexNet" — Won the ILSVRC2012 Challenge

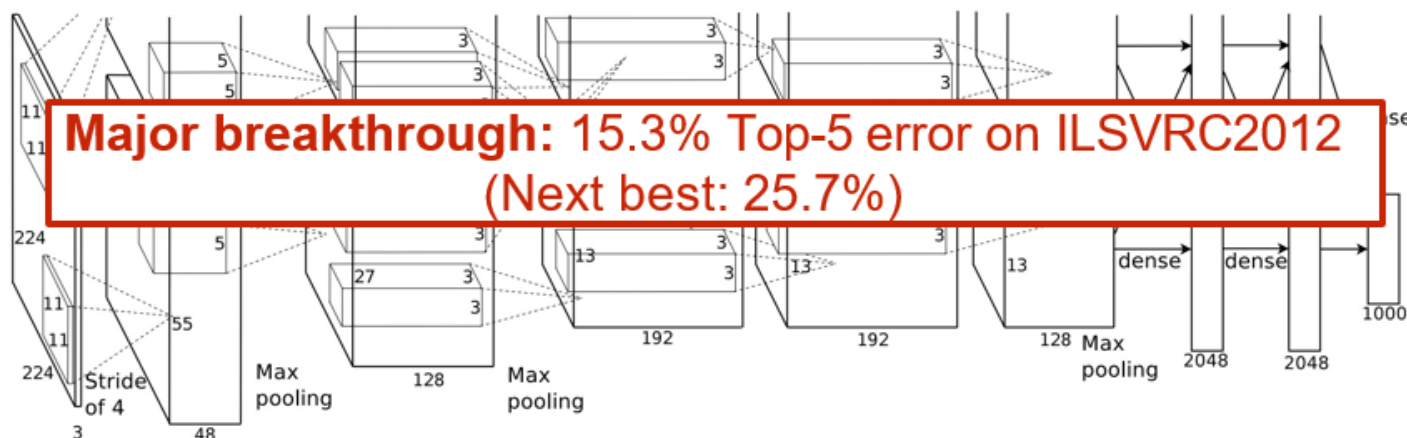
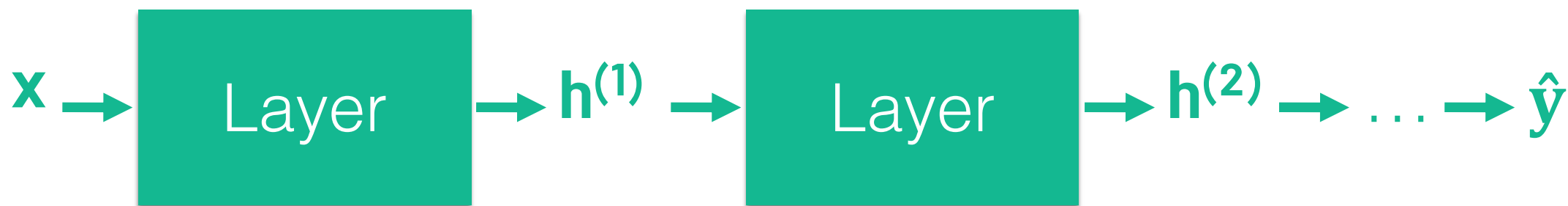


Figure 2: An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

[Krizhevsky, Sutskever, Hinton. NIPS 2012]



What Shape should the activations have?

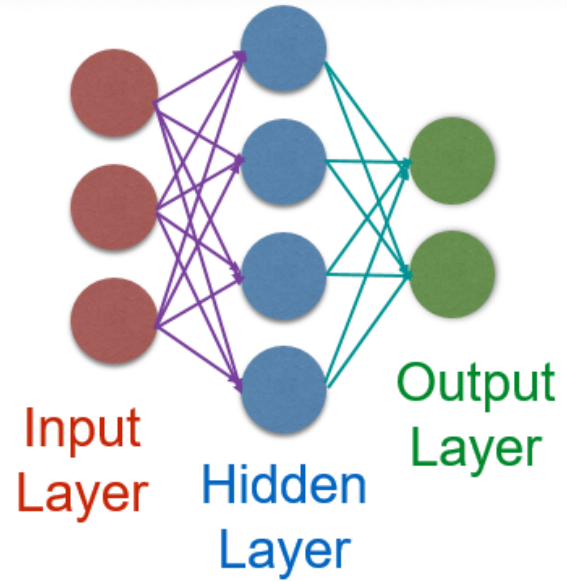


- The input is an image, which is 3D (RGB Channel, Height, Width)
- We could flatten into a 1D vector, but then we lose structure
- What about keeping everything in 3D?

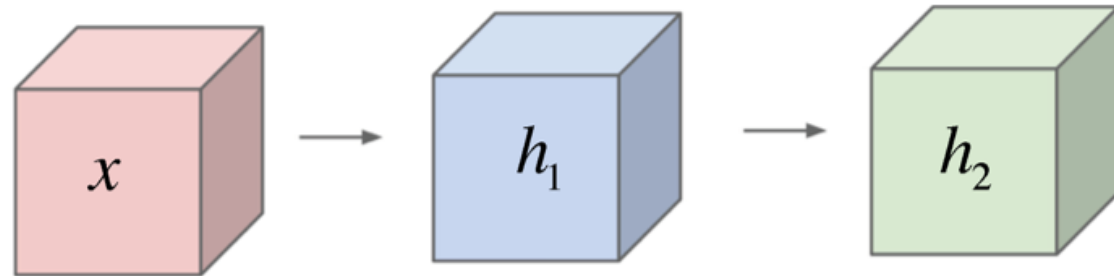


3D Activations

Before:



now:

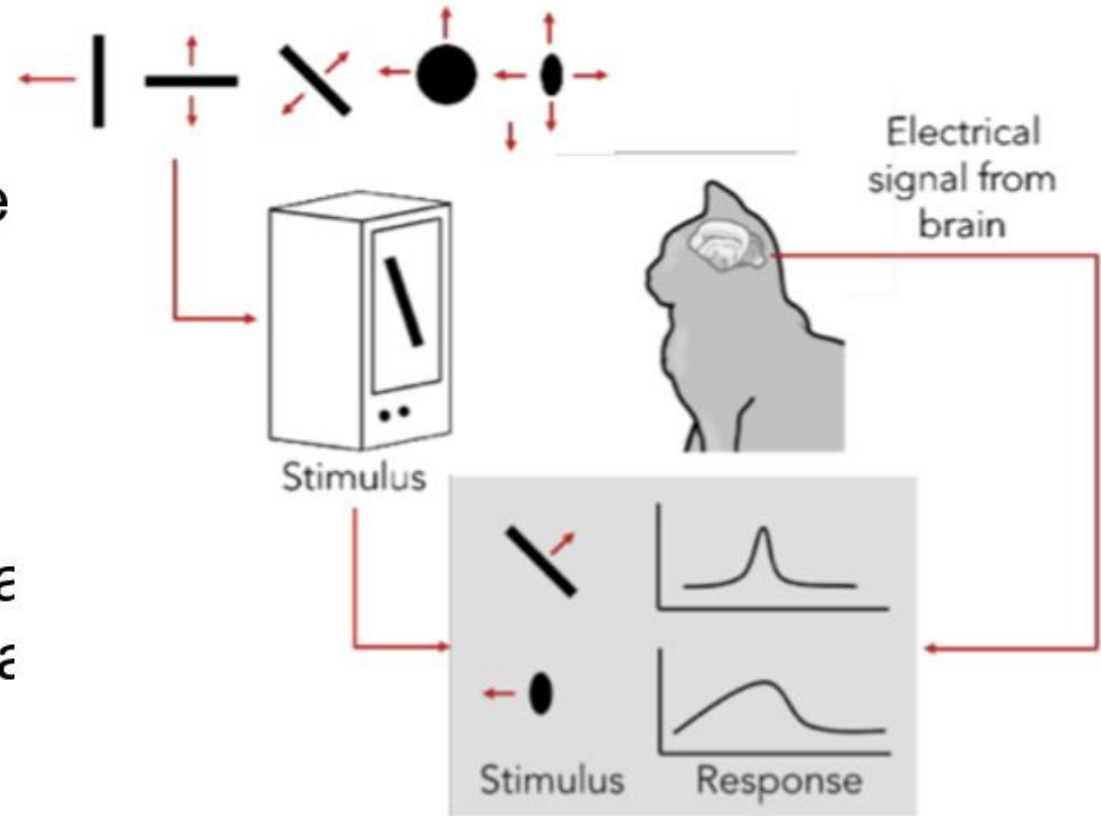


(3D arrays)



Biological Motivation

- **Hubel & Wiesel 1959**
 - Receptive fields of single neurons
- **1962**
 - Receptive fields, binocular interactions and functional architecture
- ...





3D Activations

All Neural Net
activations
arranged in **3
dimensions:**

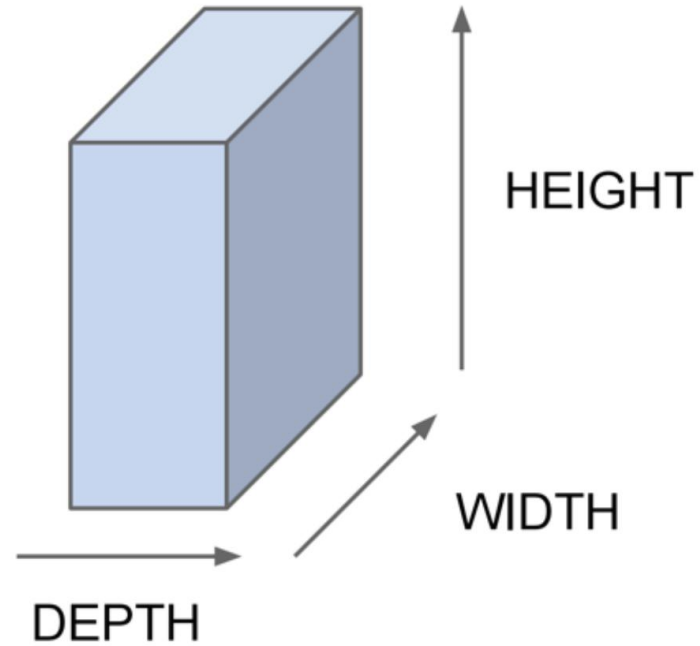


Figure: Andrej Karpathy

Convolutional NNs

- Basics of CNNs
- Modern CNNs
- Batch Normalization
- Transfer Learning

Basics of CNNs

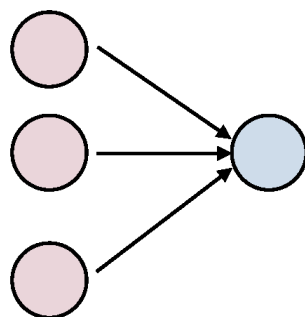
Convolutional NNs

- **Basics of CNNs**
 - Modern CNNs
 - Batch Normalization
 - Transfer Learning



3D Activations

1D Activations:



3D Activations:

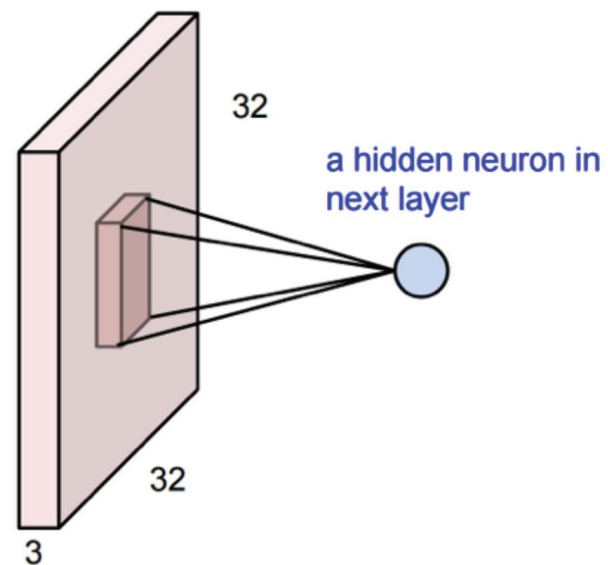
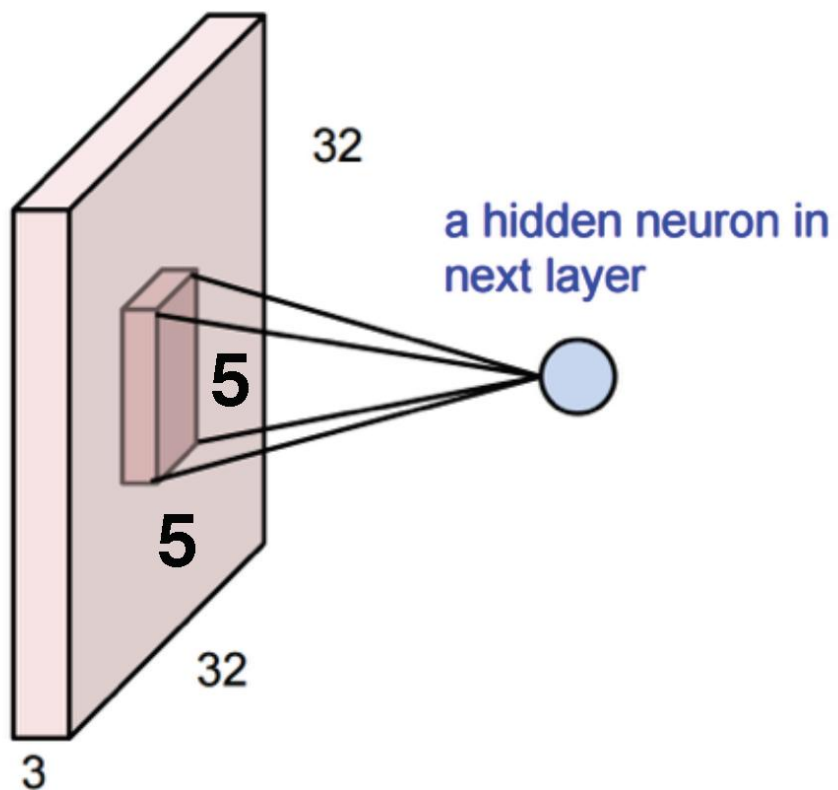


Figure: Andrej Karpathy



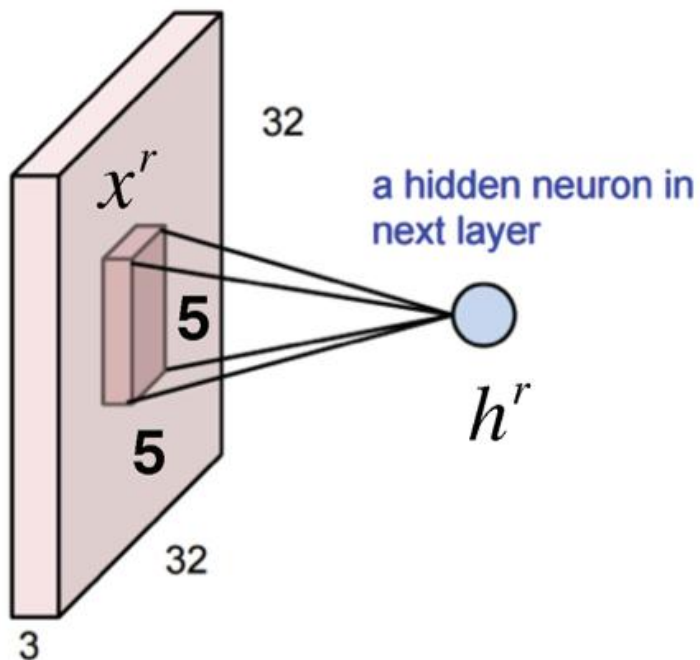
3D Activations



- The input is $3 \times 32 \times 32$
- This neuron depends on a $3 \times 5 \times 5$ chunk of the input
- The neuron also has a $3 \times 5 \times 5$ set of weights and a bias (scalar)



3D Activations



Example: consider the region of the input “ x^r ”

With output neuron h^r

Then the output is:

$$h^r = \sum_{ijk} x^r_{ijk} W_{ijk} + b$$

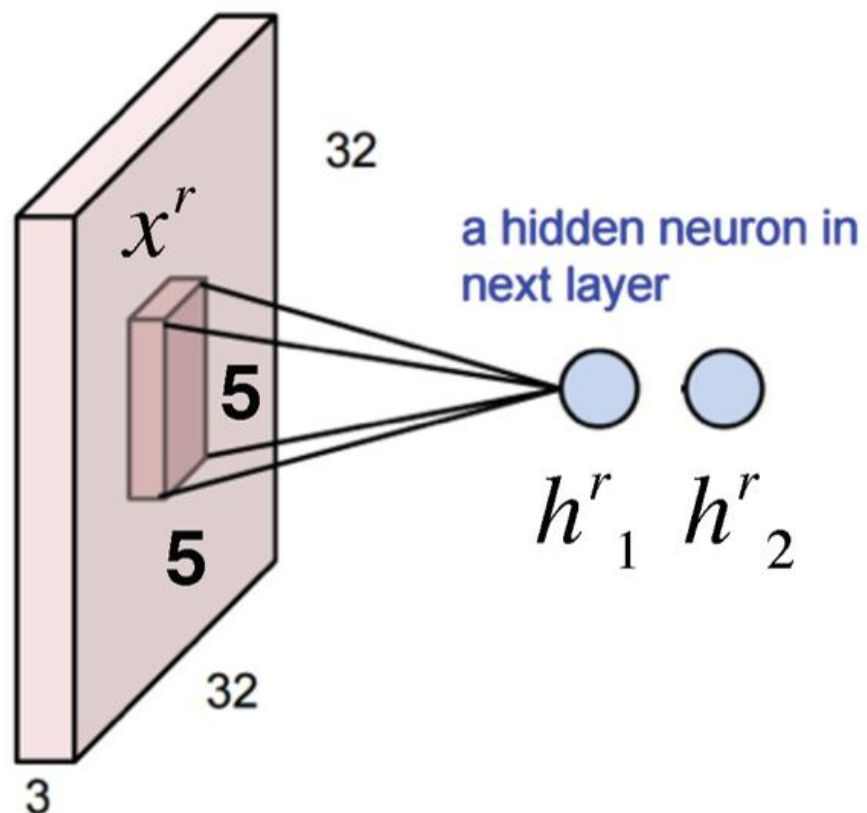


Sum over 3 axes

Figure: Andrej Karpathy



3D Activations



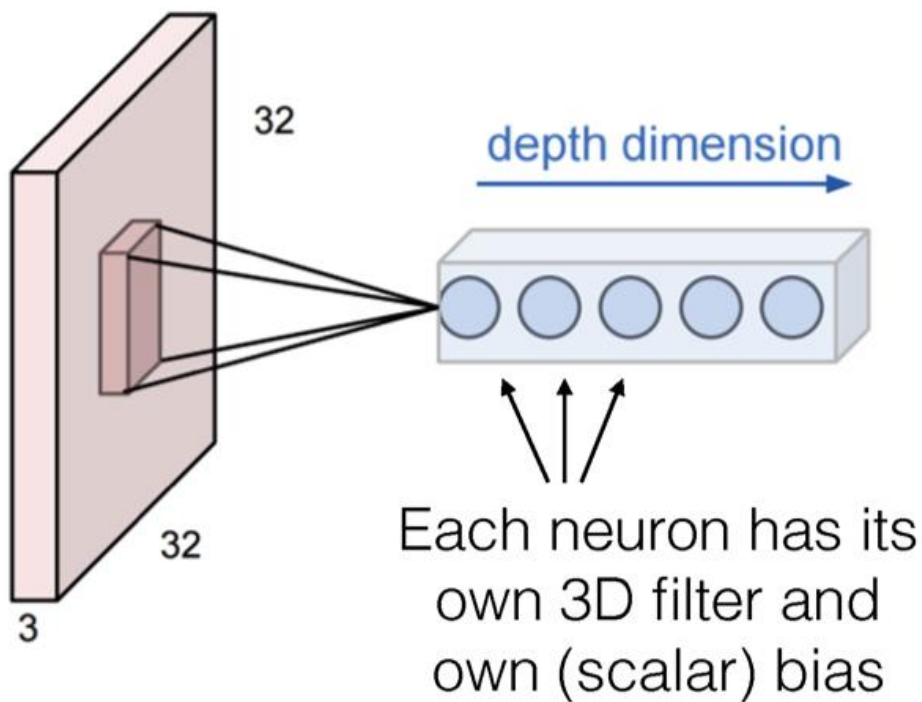
With **2** output neurons

$$h^r_1 = \sum_{ijk} x^r_{ijk} W_{1ijk} + b_{1}$$

$$h^r_2 = \sum_{ijk} x^r_{ijk} W_{2ijk} + b_{2}$$



3D Activations



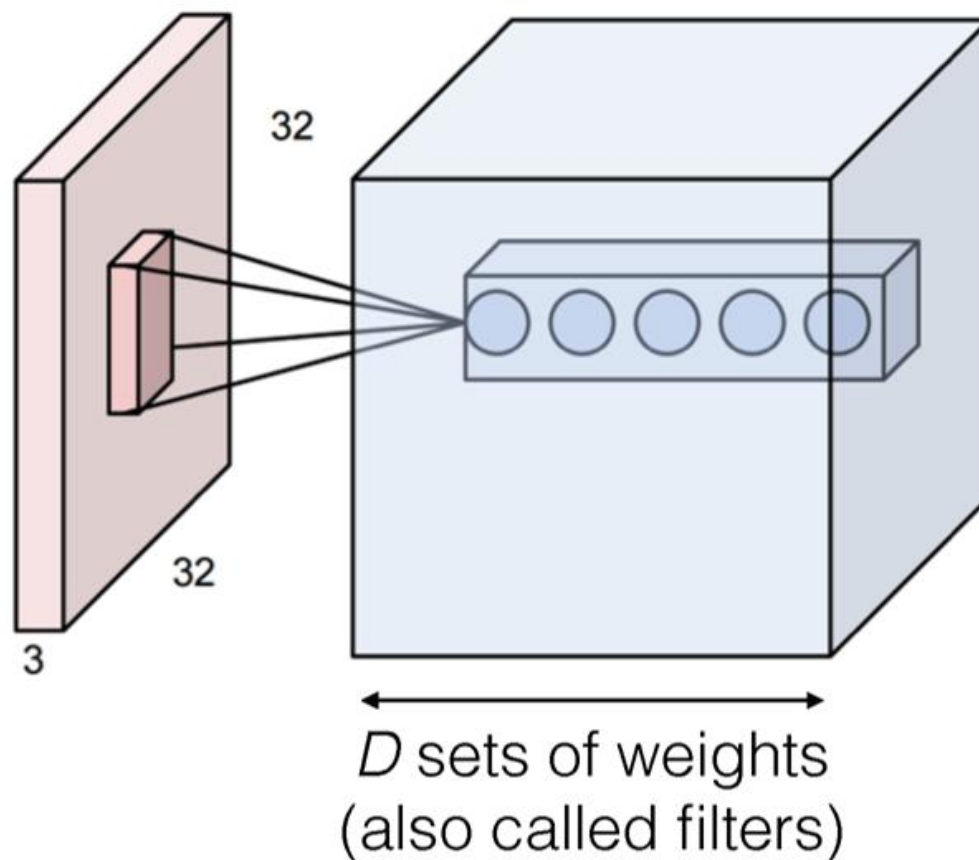
We can keep adding more outputs

These form a column in the output volume:
[depth x 1 x 1]

Figure: Andrej Karpathy



3D Activations



Now repeat this across the input

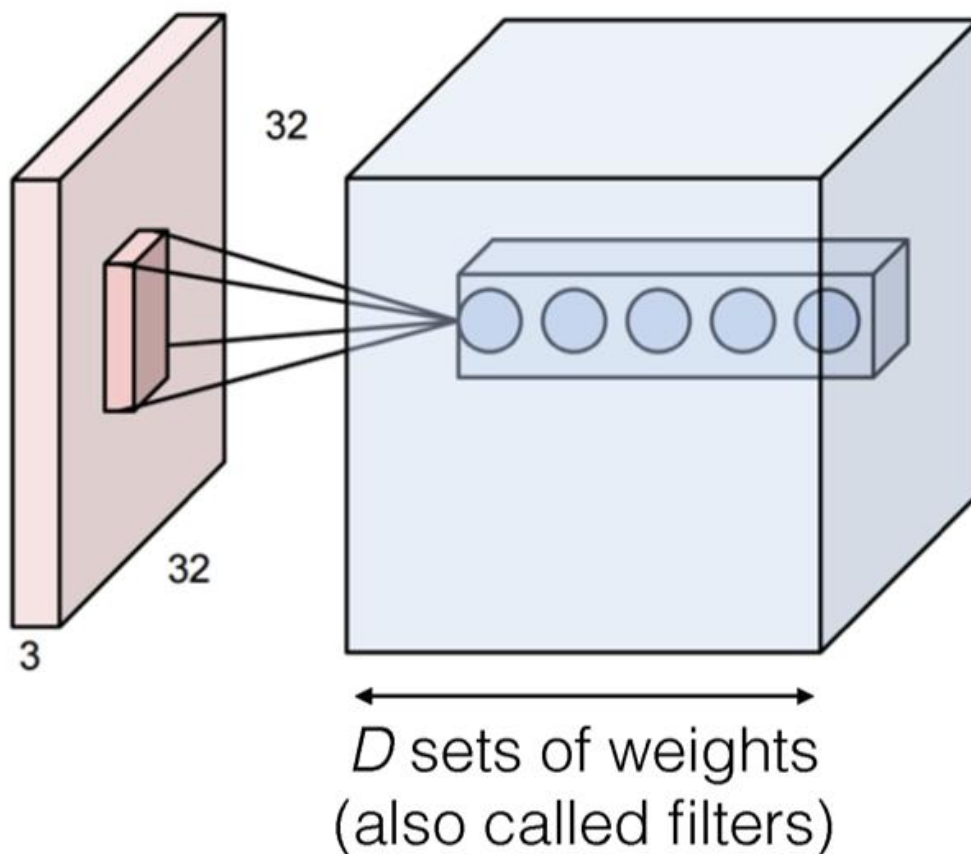
Weight sharing:

Each filter shares the same weights (but each depth index has its own set of weights)

Figure: Andrej Karpathy



3D Activations



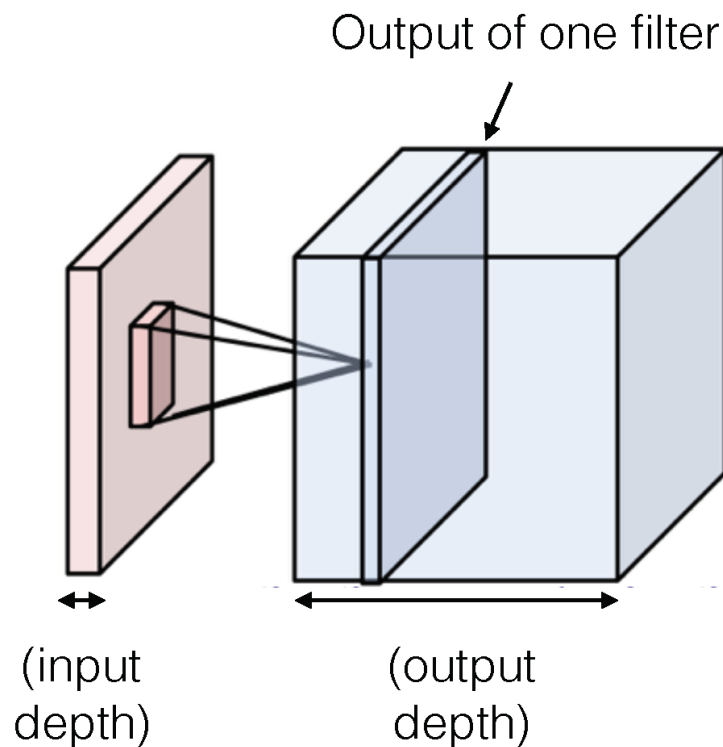
With weight sharing, this is called **convolution**

Without weight sharing, this is called a **locally connected layer**

Figure: Andrej Karpathy



Convolutional Layer



One set of weights gives one slice in the output

To get a 3D output of depth D , use D different filters

In practice, ConvNets use many filters (~ 64 to 1024)

All together, the weights are **4** dimensional:
(output depth, input depth, kernel height, kernel width)



Convolutional Layer

Example with one input channel
and zero bias

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Convolutional Layer

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10							

$$g[.,.] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$



Convolutional Layer

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20						

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$



Convolutional Layer

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30					

$$g[.,.] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$



Convolutional Layer

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



Convolutional Layer

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

	0	10	20	30	30				

$$g[.,.] \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$



$$g[\cdot, \cdot]^{\frac{1}{9}}$$

A 10x10 grid with a red box containing a question mark at row 4, column 6. The grid has numerical labels: 0, 10, 20, 30, 30 in the first row; 50 in the fourth row, fifth column.

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$



Convolutional Layer

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

		0	10	20	30	30	30	20	10
		0	20	40	60	60	60	40	20
		0	30	60	90	90	90	60	30
		0	30	50	80	80	90	60	30
		0	30	50	80	80	90	60	30
		0	20	30	50	50	60	40	20
		10	20	30	30	30	30	20	10
		10	10	10	0	0	0	0	0

$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$



Convolution + Padding

Padding helps in avoiding getting smaller-sized output

$f[.,.]$

0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	90	0	90	90	90	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	90	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0

Pad with
zeros

$g[.,.] \frac{1}{9}$

1	1	1
1	1	1
1	1	1

$h[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	10	20	30	30	30	20	10	0
0	0	20	40	60	60	60	40	20	0
0	0	30	60	90	90	90	60	30	0
0	0	30	50	80	80	90	60	30	0
0	0	30	50	80	80	90	60	30	0
0	0	20	30	50	50	60	40	20	0
0	10	20	30	30	30	30	20	10	0
0	10	10	10	0	0	0	0	0	0
0	10	10	10	0	0	0	0	0	0

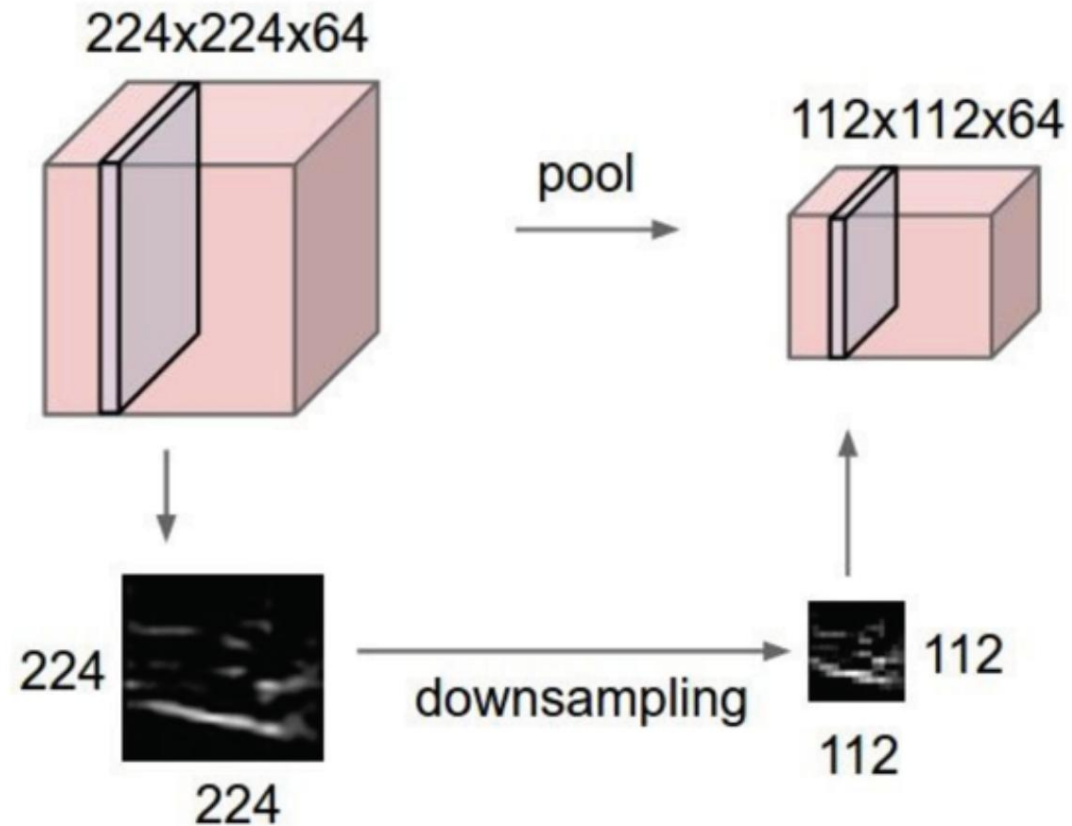
$$h[m,n] = \sum_{k,l} g[k,l] f[m+k,n+l]$$

Pooling in CNNs



Pooling

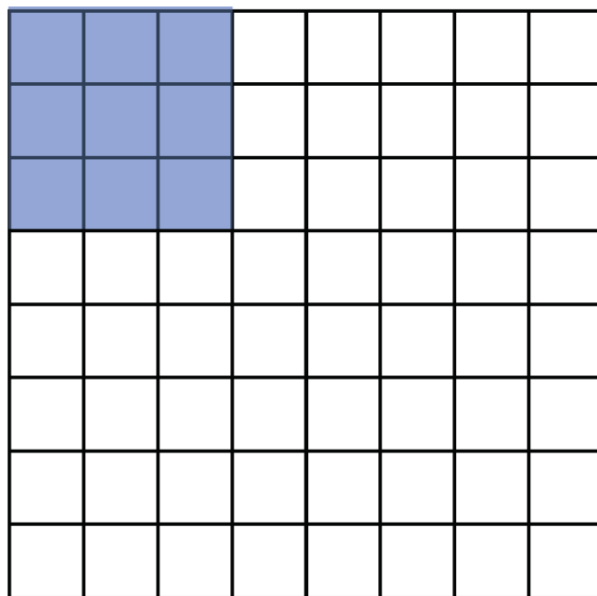
- Makes the representation smaller and more manageable
- Operates over each activation map independently





Stride in Convolution = Pooling

During convolution, the weights “slide” along the input to generate each output



Input

Recall that at each position, we are doing a **3D** sum:

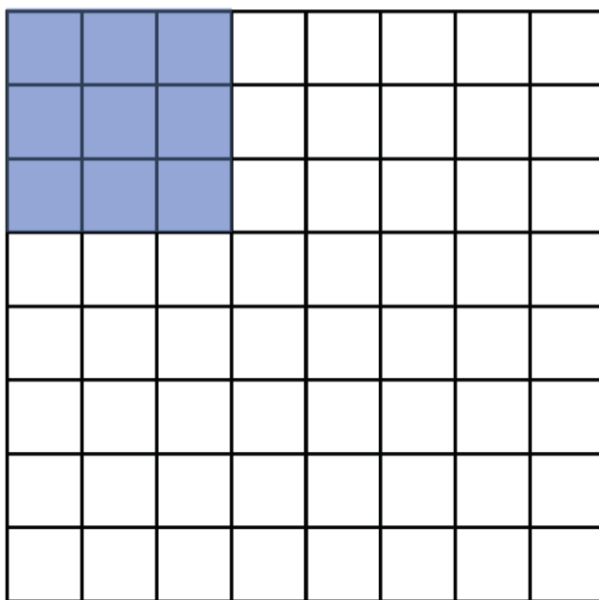
$$h^r = \sum_{ijk} x^r_{ijk} W_{ijk} + b$$

(channel, row, column)

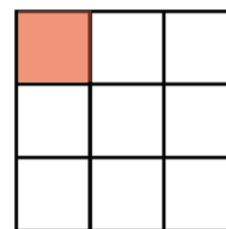


Stride in Convolution

But we can also convolve with a **stride**, e.g. stride = 2



Input

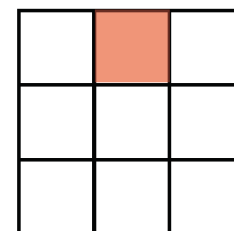
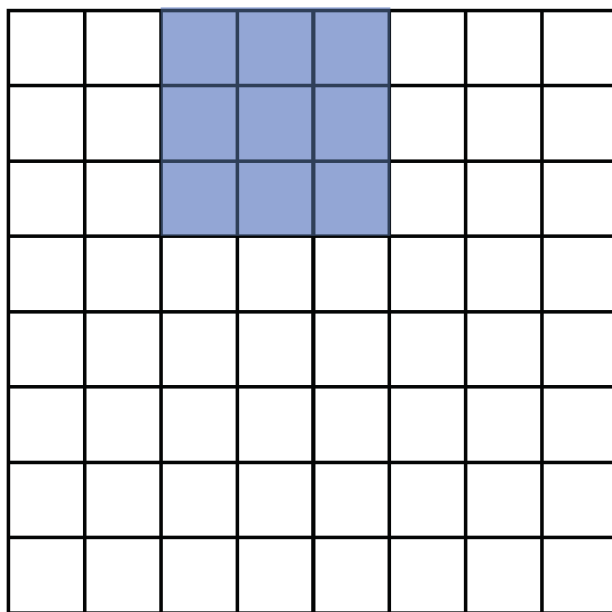


Output



Stride in Convolution

But we can also convolve with a **stride**, e.g. stride = 2



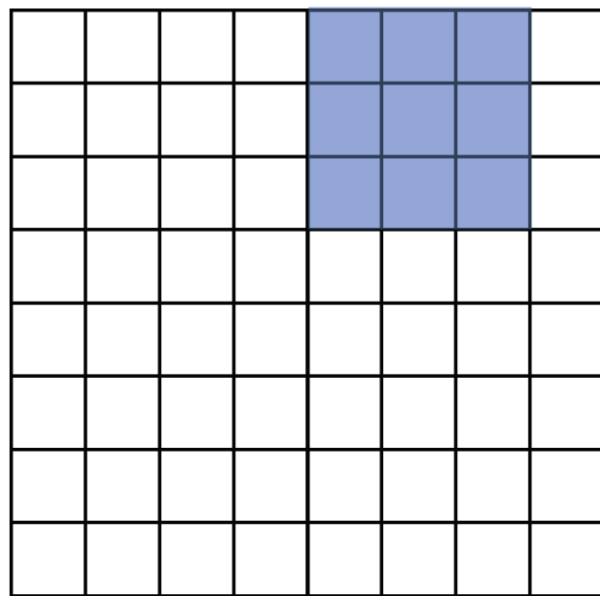
Output

Input

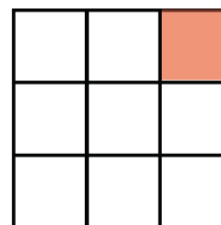


Stride in Convolution

But we can also convolve with a **stride**, e.g. stride = 2



Input



Output



Stride + Padding

We can also pad the input with zeros.

Here, **pad = 1**, **stride = 2**

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input

Output



Stride + Padding

We can also pad the input with zeros.
Here, **pad = 1**, **stride = 2**

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input

Output



Stride + Padding

We can also pad the input with zeros.

Here, **pad = 1**, **stride = 2**

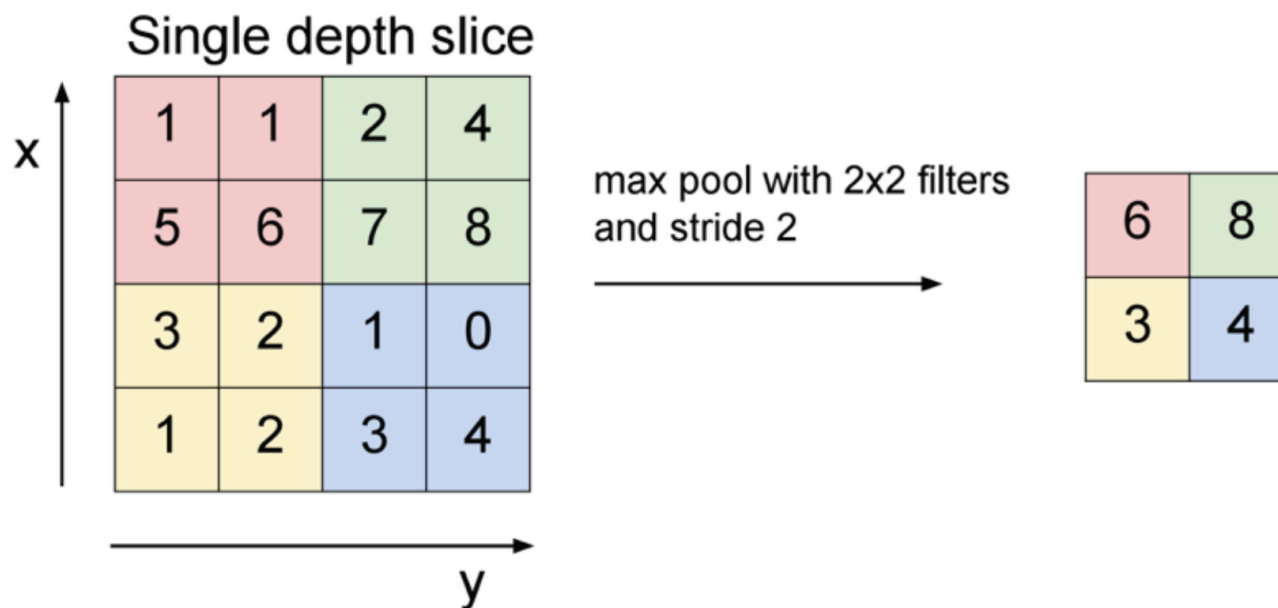
0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input

Output



Max Pooling + Stride

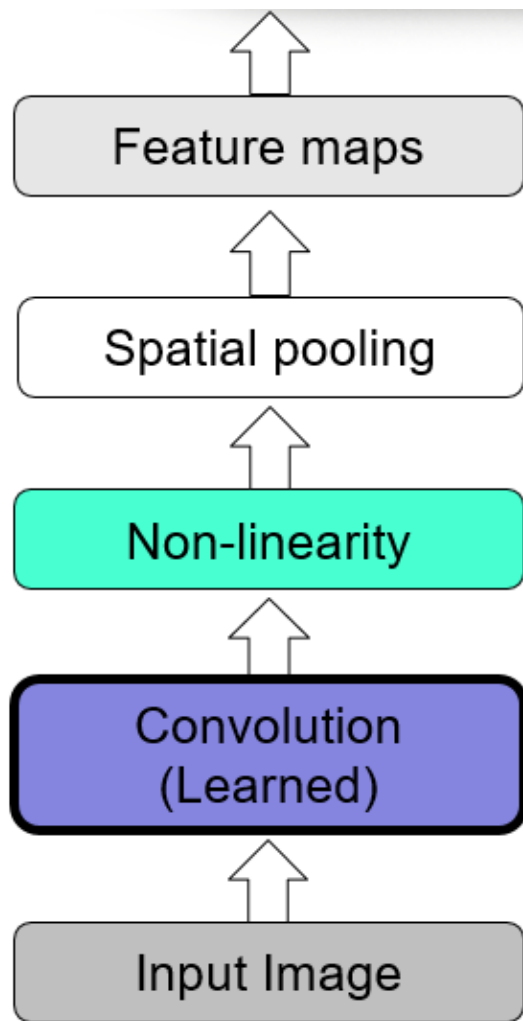


What's the backprop rule for max pooling?

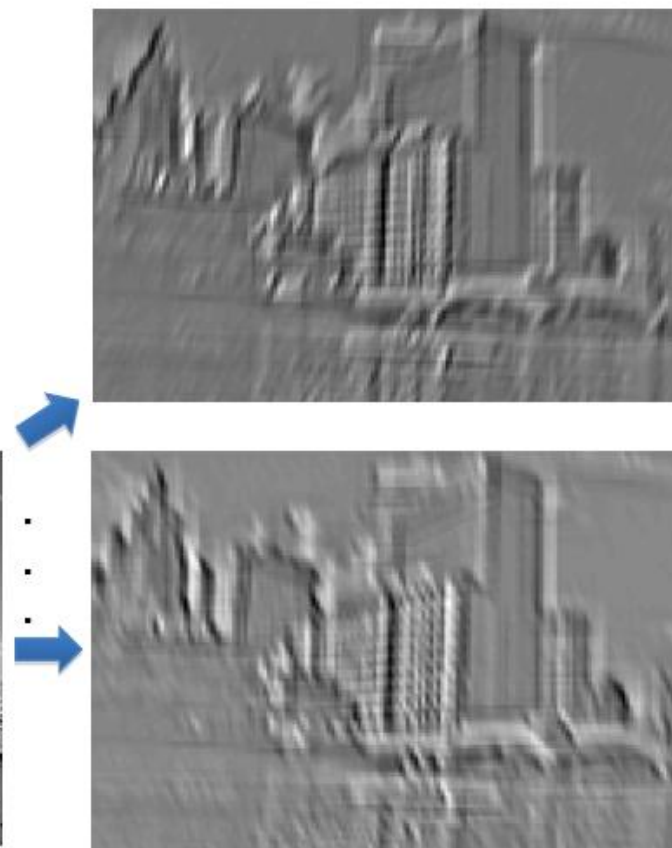
- In the forward pass, store the index that took the max
- The backprop gradient is the input gradient at that index



Operations in CNNs



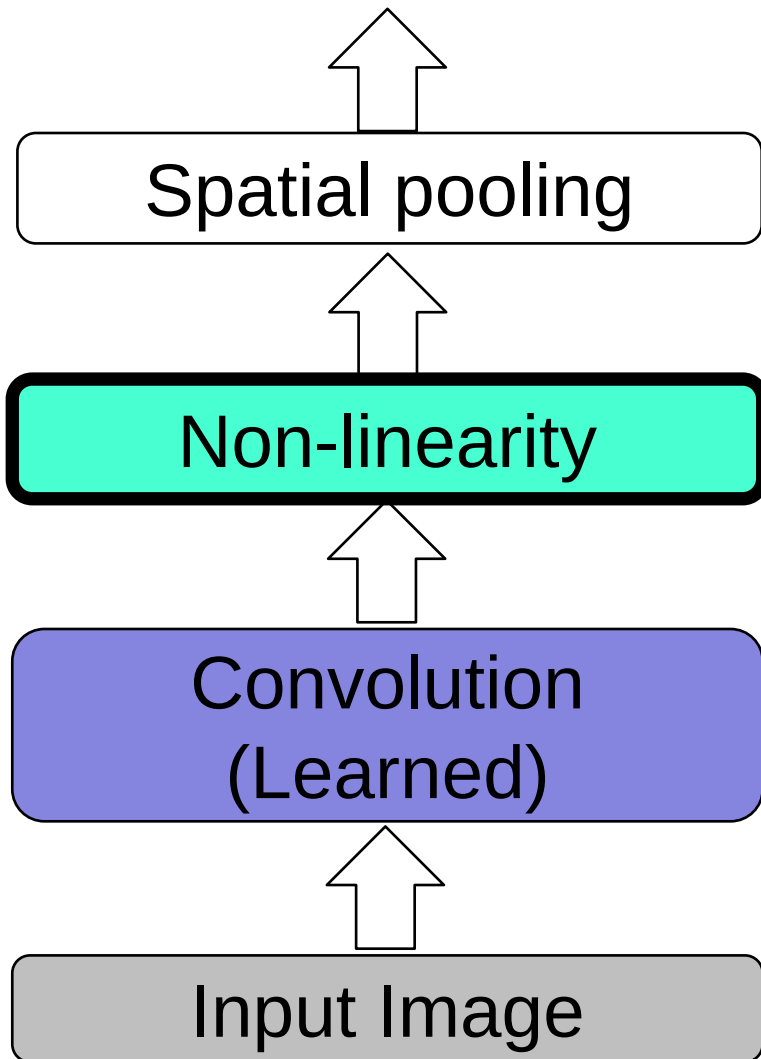
Input



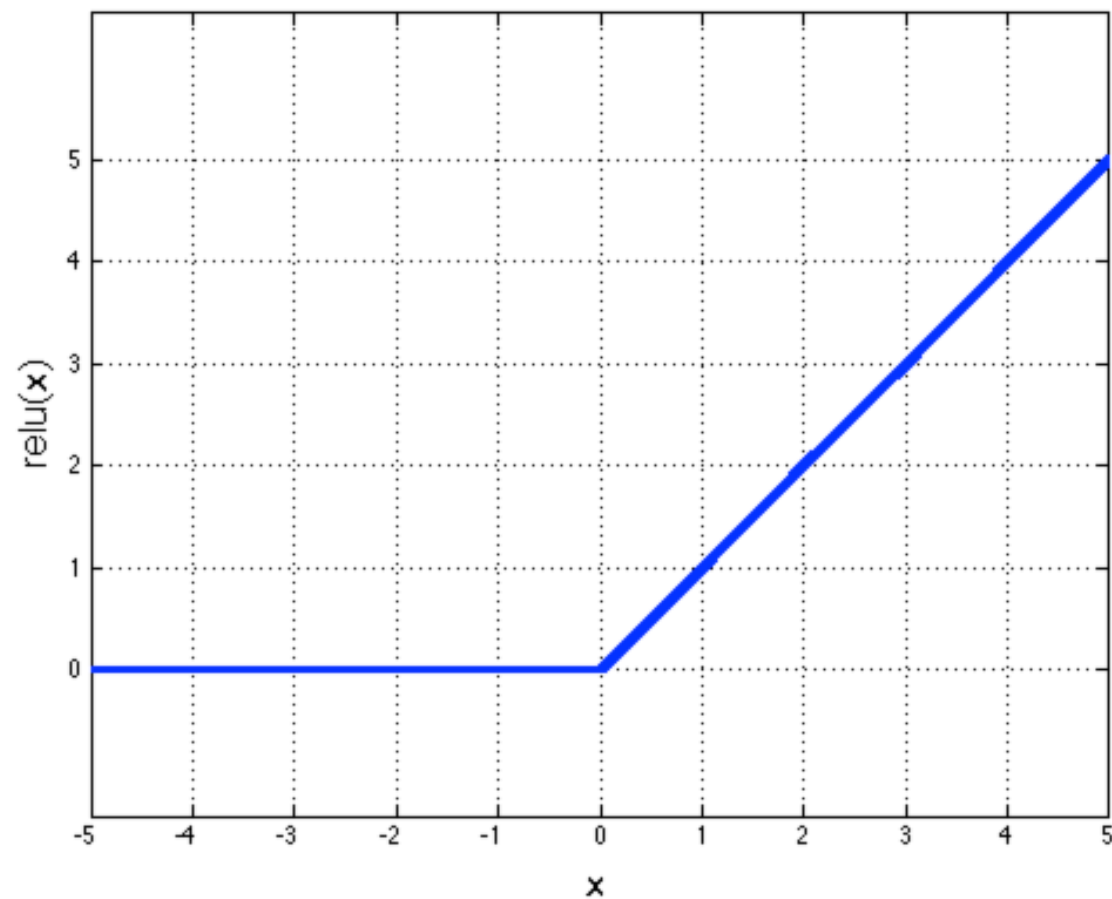
Feature Map



Operations in CNNs

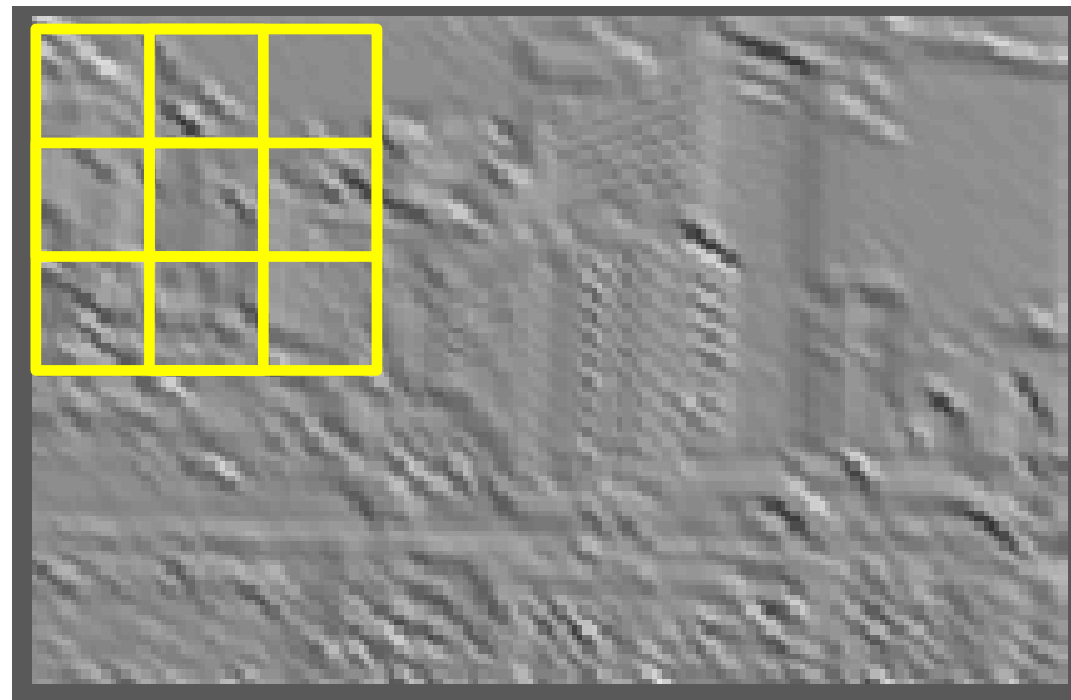
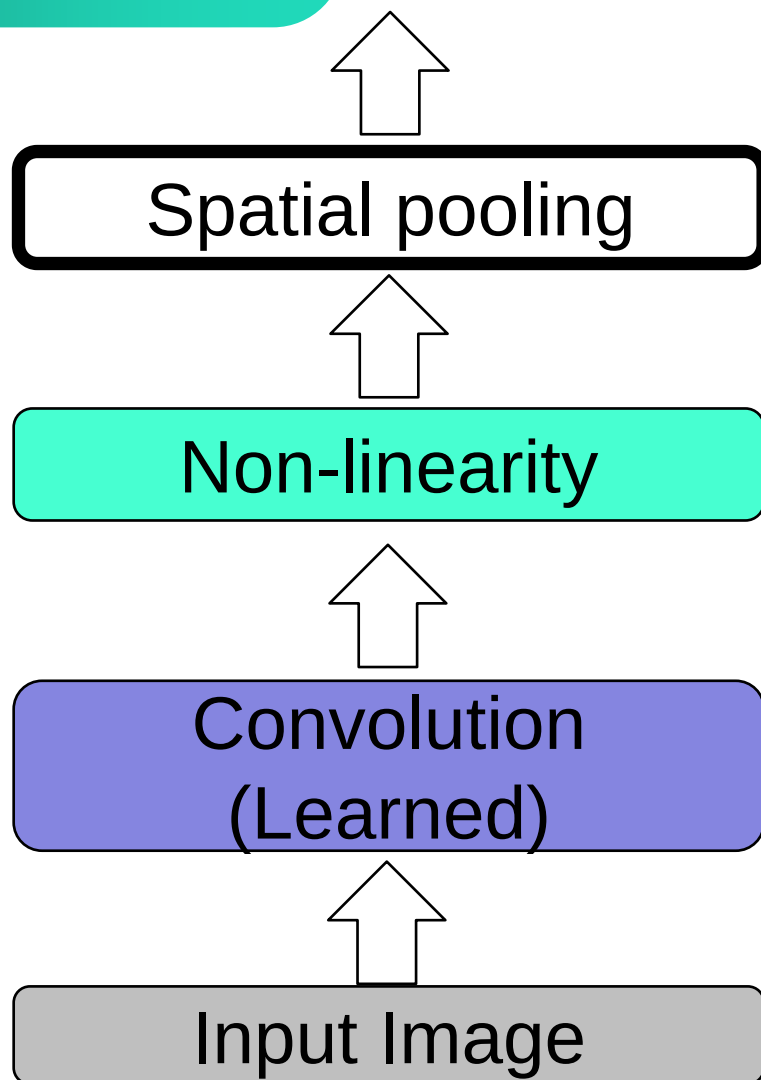


Rectified Linear Unit (ReLU)

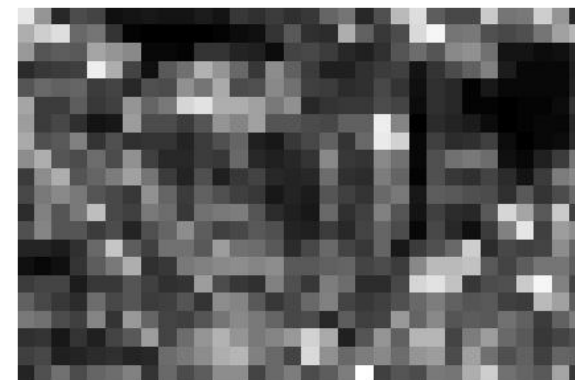


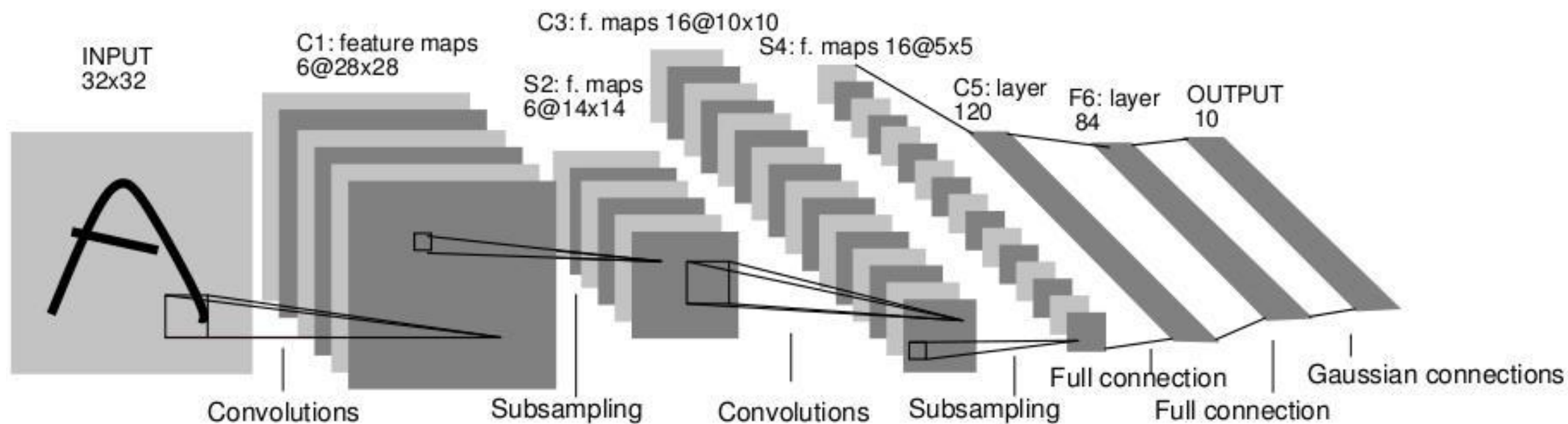


Operations in CNNs



Max





Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, [Gradient-based learning applied to document recognition](#), Proc. IEEE 86(11): 2278–2324, 1998.



From Local Simple to Complex Global Features

Preview

[Zeiler and Fergus 2013]

Visualization of VGG-16 by Lane McIntosh. VGG-16 architecture from [Simonyan and Zisserman 2014].

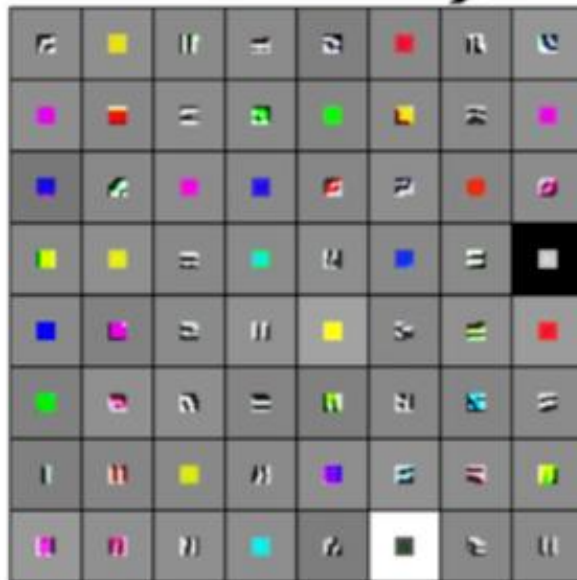


Low-level features

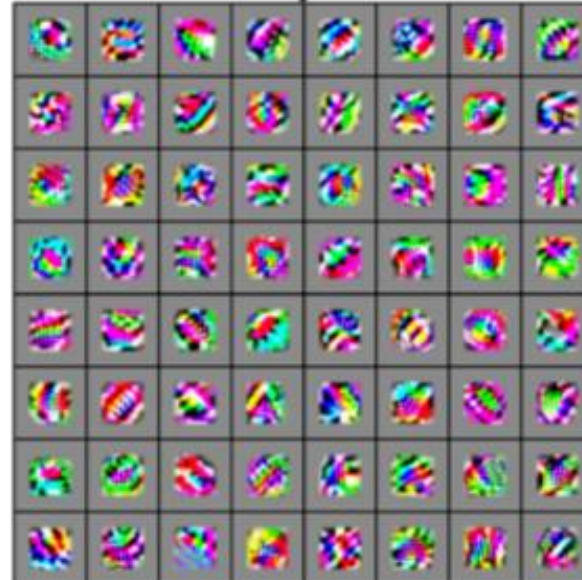
Mid-level features

High-level features

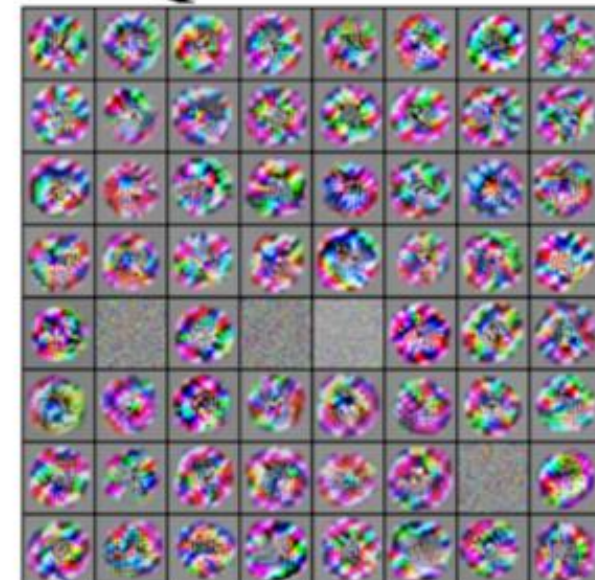
Linearly separable classifier



VGG-16 Conv1_1



VGG-16 Conv3_2

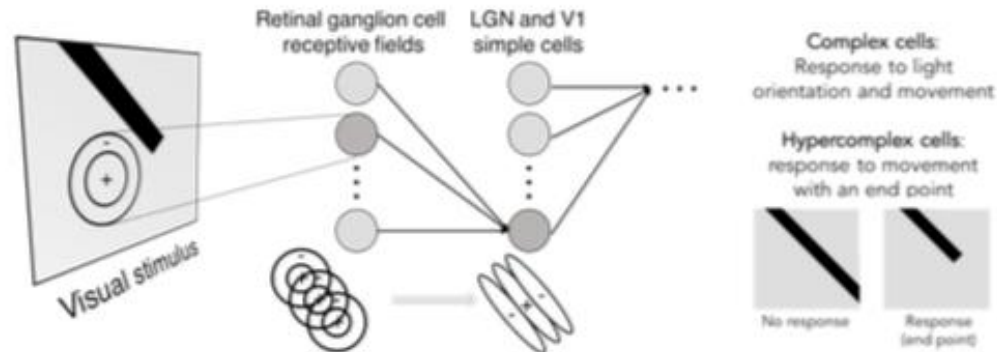
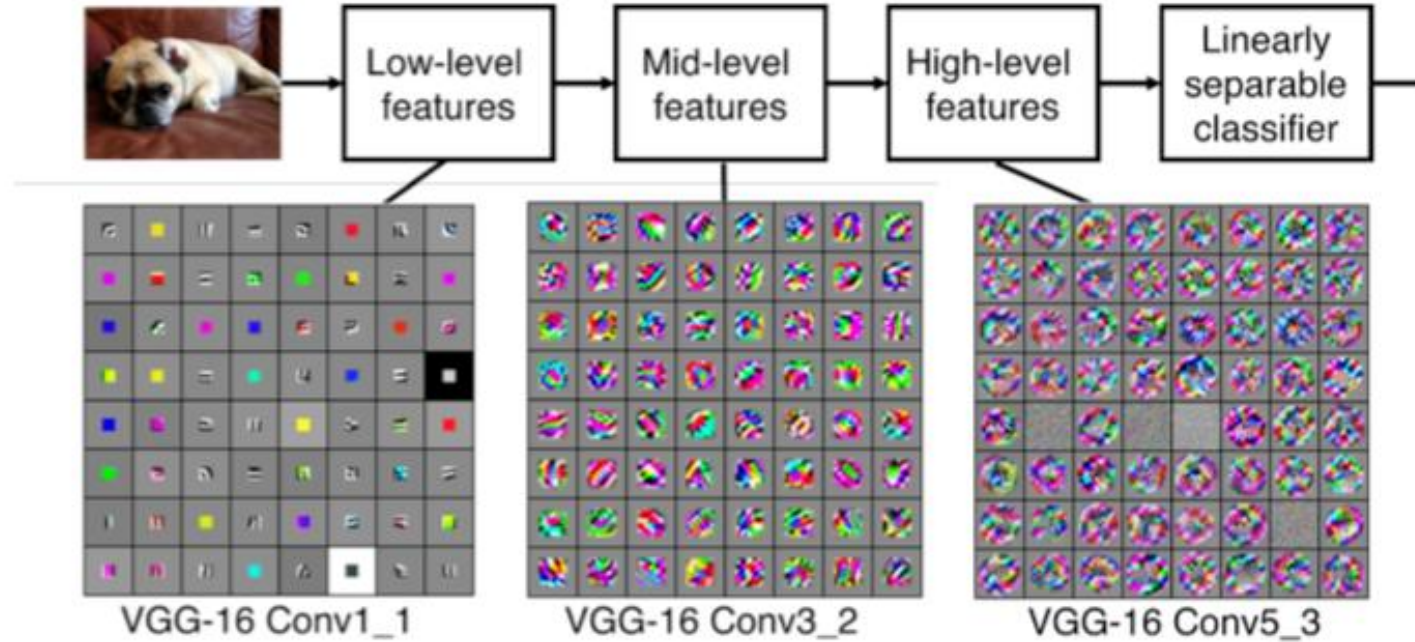


VGG-16 Conv5_3



From Local Simple to Complex Global Features

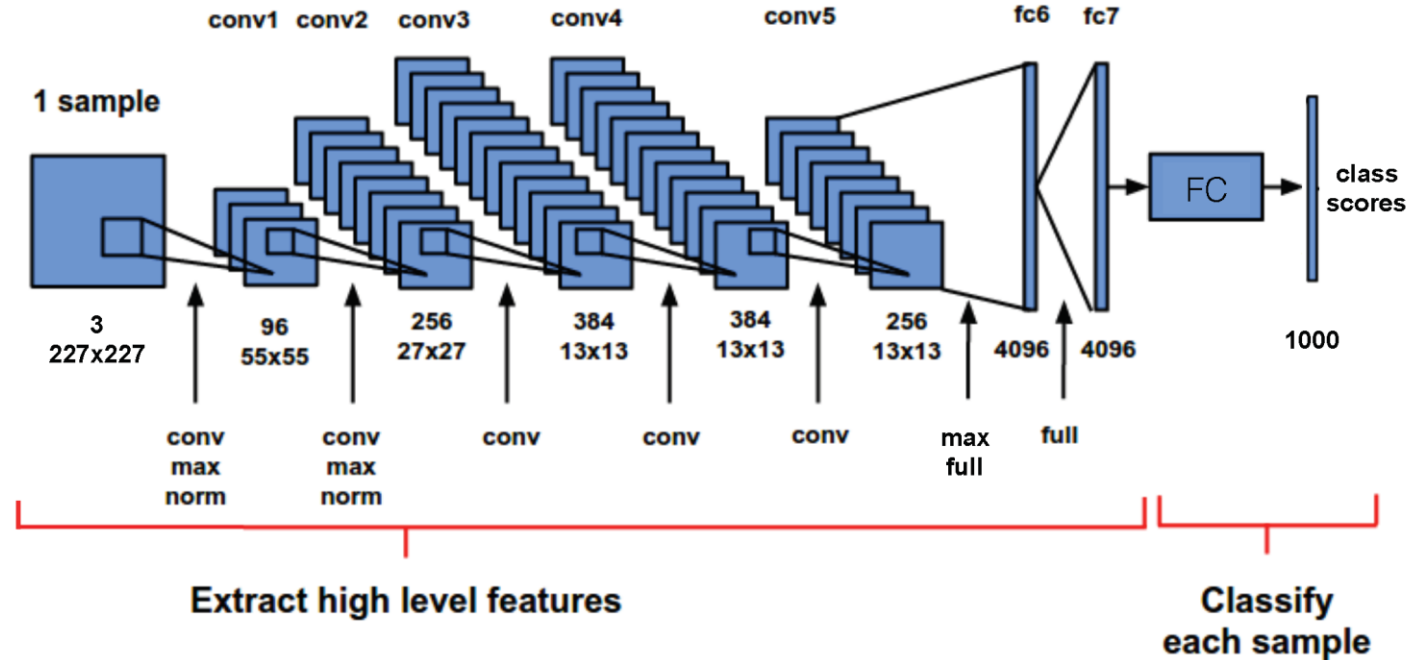
Preview





Example: AlexNet [Krizhevsky 2012]

Example: AlexNet [Krizhevsky 2012]



“max”: max pooling

“norm”: local response normalization

“full”: fully connected

Figure: [Karnowski 2015] (with corrections)

2 DNN

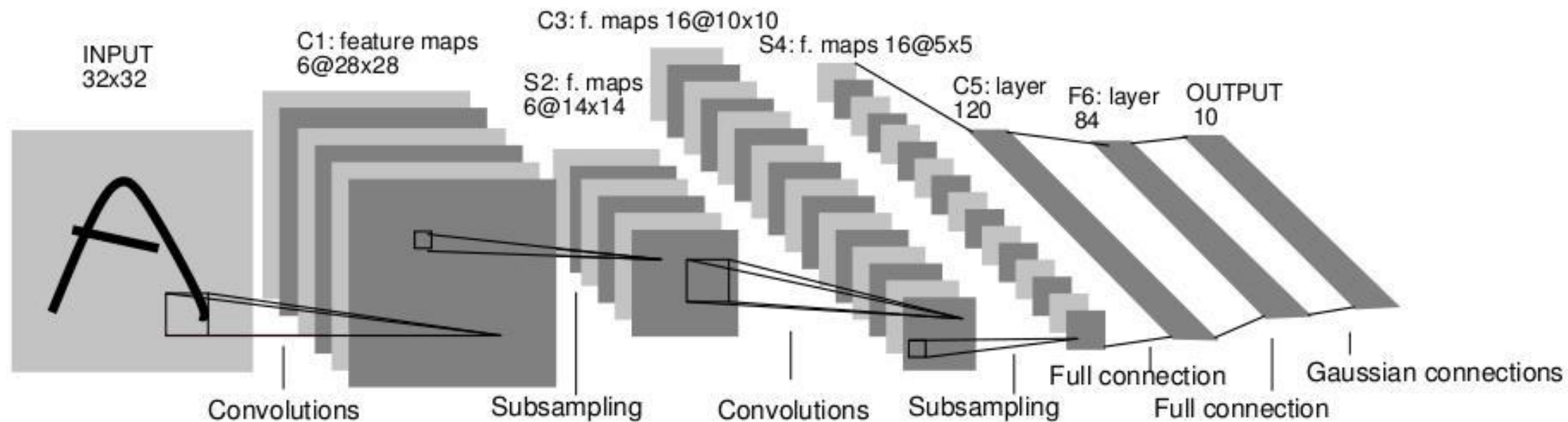
Convolutional NNs

- Basics of CNNs
- **Modern CNNs**
- Batch Normalization
- Transfer Learning



First Convolutional NN

LeNet 5

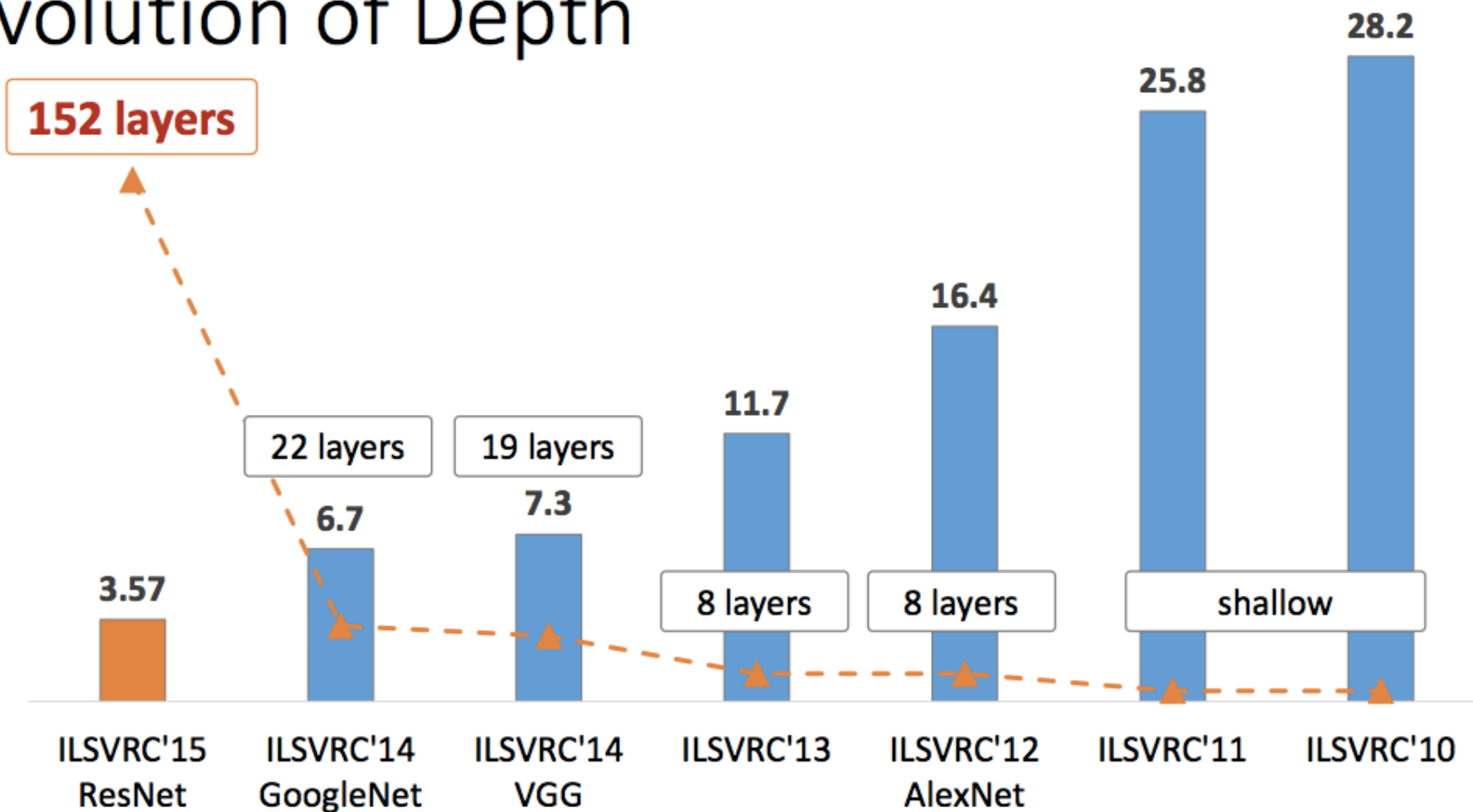


Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, [Gradient-based learning applied to document recognition](#), Proc. IEEE 86(11): 2278–2324, 1998.



Rapid Improvement of CNN Architectures

Revolution of Depth

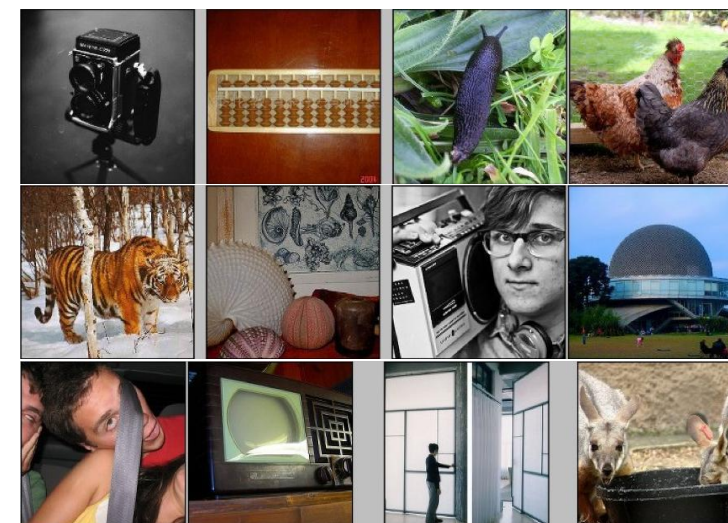




Arrival of Big Visual Data

- 14 million labeled images, 20k classes
- Images gathered from Internet
- Human labels via Amazon MTurk
- ImageNet Large-Scale Visual Recognition Challenge (ILSVRC):
1.2 million training images, 1000 classes

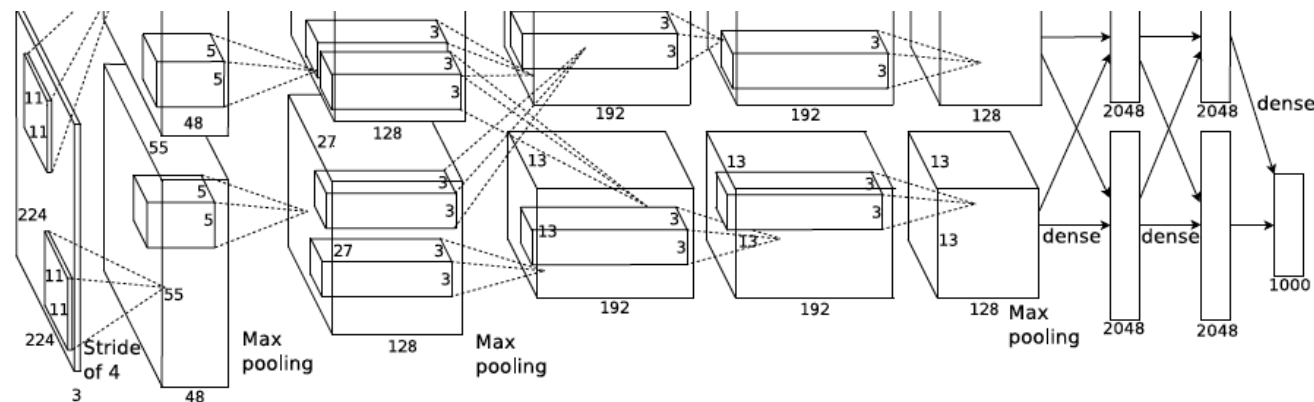
IMAGENET



www.image-net.org/challenges/LSVRC/



AlexNet: ILSVRC 2012 winner



- Similar framework to LeNet but:
 - Max pooling, ReLU nonlinearity, Dropout regularization
 - More data and bigger model (7 hidden layers, 650K units, 60M params)
 - GPU implementation (50x speedup over CPU)

A. Krizhevsky, I. Sutskever, and G. Hinton, [ImageNet Classification with Deep Convolutional Neural Networks](#), NIPS 2012



Clarifai: ILSVRC 2013 winner

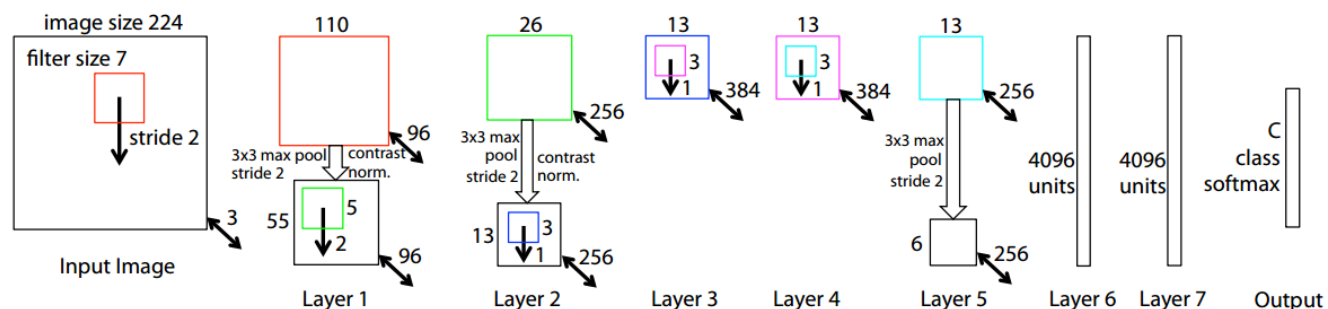


Figure 3. Architecture of our 8 layer convnet model. A 224 by 224 crop of an image (with 3 color planes) is presented as the input. This is convolved with 96 different 1st layer filters (red), each of size 7 by 7, using a stride of 2 in both x and y. The resulting feature maps are then: (i) passed through a rectified linear function (not shown), (ii) pooled (max within 3x3 regions, using stride 2) and (iii) contrast normalized across feature maps to give 96 different 55 by 55 element feature maps. Similar operations are repeated in layers 2,3,4,5. The last two layers are fully connected, taking features from the top convolutional layer as input in vector form ($6 \cdot 6 \cdot 256 = 9216$ dimensions). The final layer is a C -way softmax function, C being the number of classes. All filters and feature maps are square in shape.

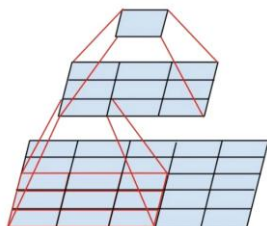
➤ Refinement of Alexnet

M. Zeiler and R. Fergus, [Visualizing and Understanding Convolutional Networks](#), ECCV 2014 (Best Paper Award winner)



Data Engineering Skills

- Sequence of deeper networks trained progressively
- Large receptive fields replaced by successive layers of 3x3 convolutions (with ReLU in between)



- One 7x7 conv layer with C feature maps needs $49C^2$ weights, three 3x3 conv layers need only $27C^2$ weights
- Experimented with 1x1 convolutions

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

K. Simonyan and A. Zisserman, [Very Deep Convolutional Networks for Large-Scale Image Recognition](#), ICLR 2015

Residual Connection



ResNet: ILSVRC 2015 winner

Revolution of Depth

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)



ResNet, 152 layers
(ILSVRC 2015)



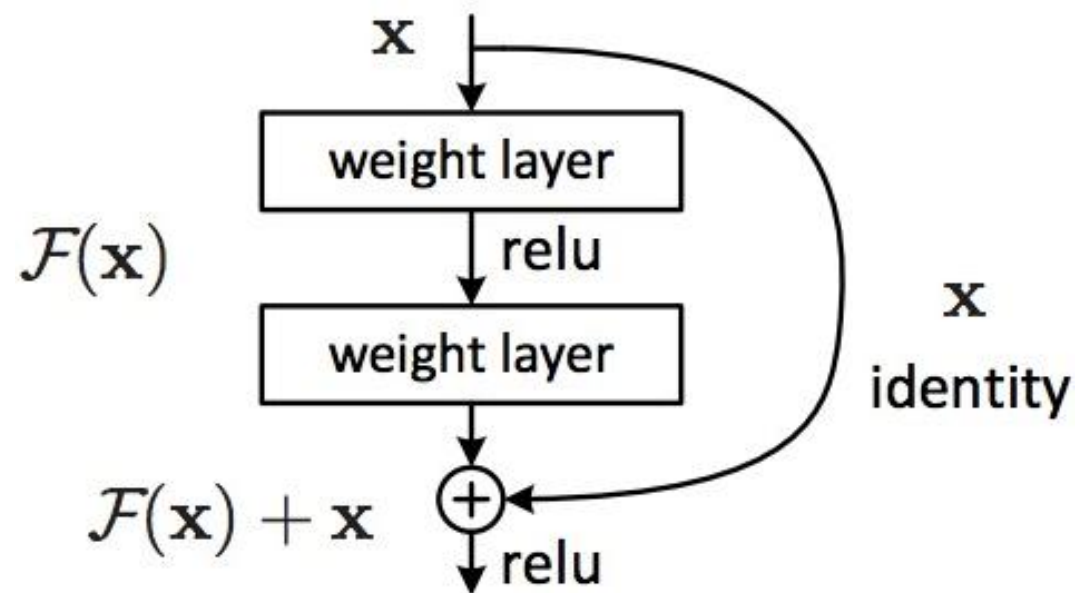
ICCV15

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016



ResNet: ILSVRC 2015 winner

- The residual module
- Introduce **skip or shortcut connections** (existing before in various forms in literature)
- Make it easy for network layers to represent the identity mapping
- For some reason, need to skip at least two layers



$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x})$$

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016

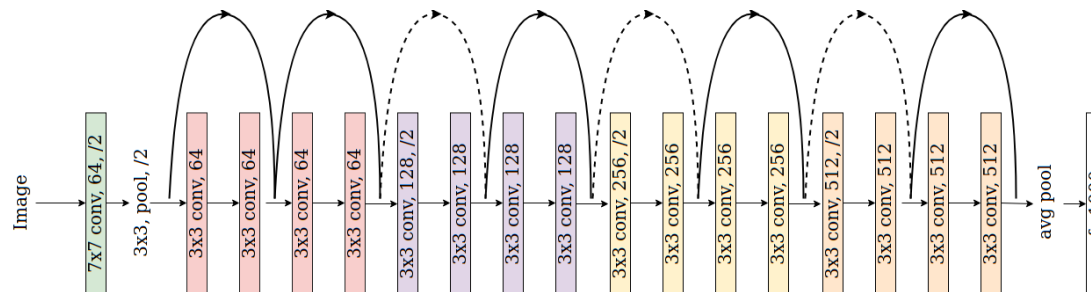


ResNet: ILSVRC 2015 winner

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016

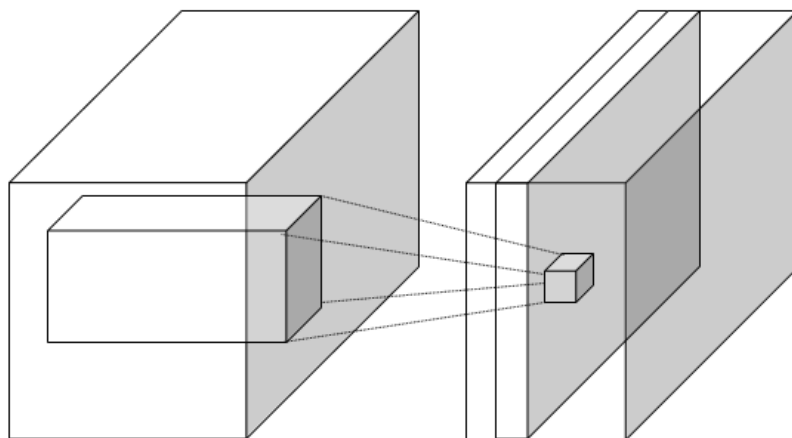
RESNET-18:



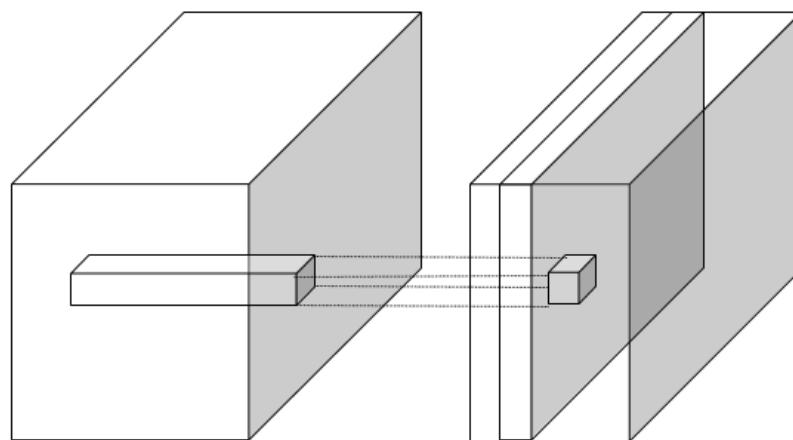
1x1 Convolution



1x1 vs larger convolutions



1x1 Convolution

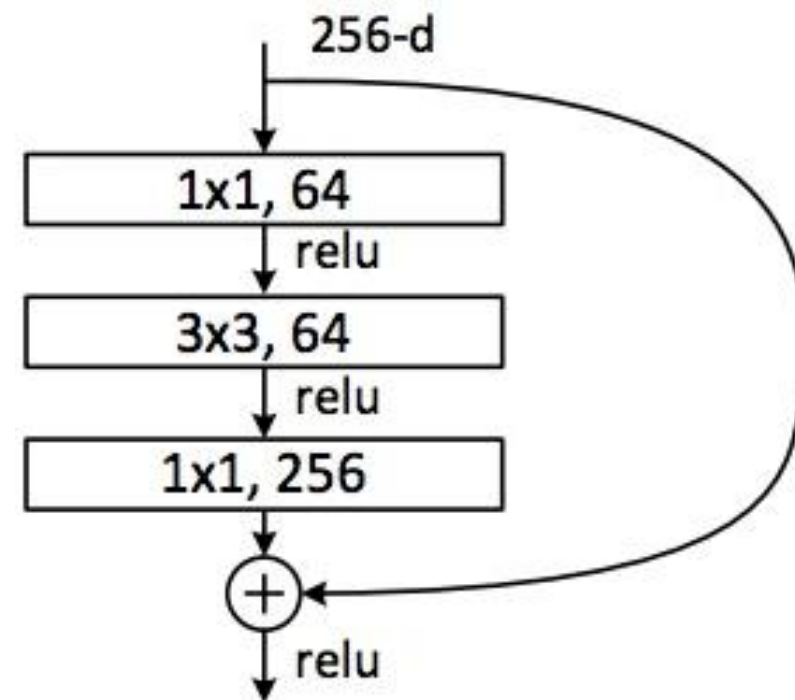




ResNet: ILSVRC 2015 winner

- ▶ Directly performing 3×3 convolutions with 256 feature maps at input and output:
 $256 \times 256 \times 3 \times 3 \sim 600\text{K}$ operations
- ▶ Using 1×1 convolutions to reduce 256 to 64 feature maps, followed by 3×3 convolutions, followed by 1×1 convolutions to expand back to 256 maps:
 $256 \times 64 \times 1 \times 1 \sim 16\text{K}$
 $64 \times 64 \times 3 \times 3 \sim 36\text{K}$
 $64 \times 256 \times 1 \times 1 \sim 16\text{K}$
Total: $\sim 70\text{K}$

Deeper residual module (bottleneck)



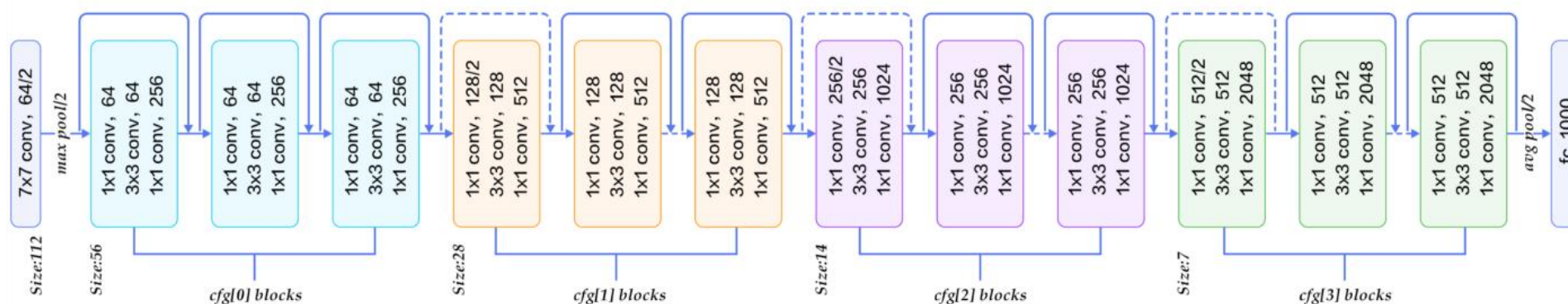
Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016



ResNet: ILSVRC 2015 winner

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

50 layers $cfg=[3,4,6,3]$
 101 layers $cfg=[3,4,23,8]$
 152 layers $cfg=[3,8,36,3]$

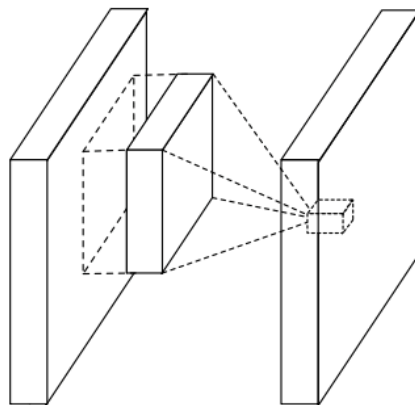


Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016

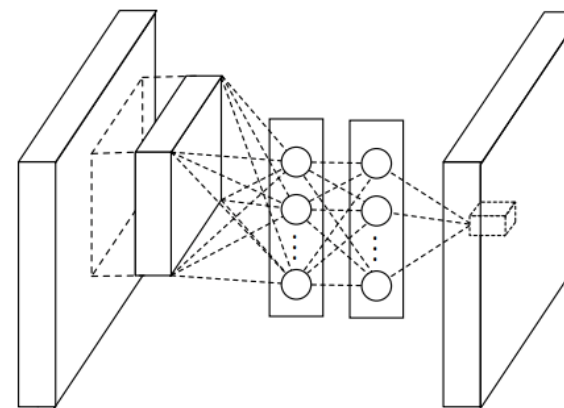
Multiple Paths



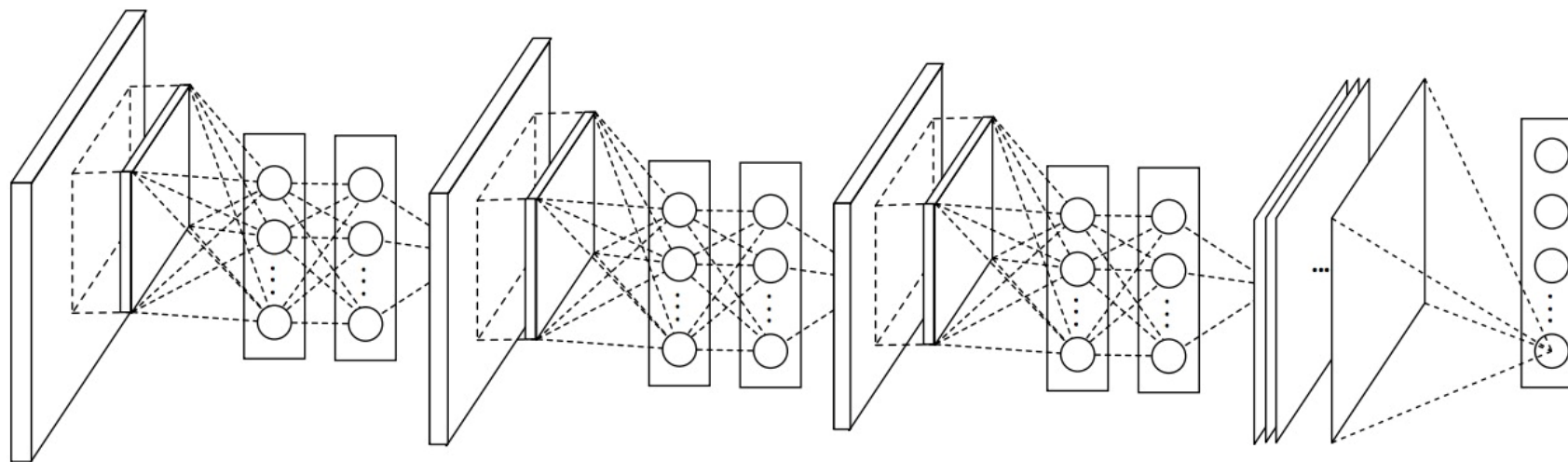
Network in network



(a) Linear convolution layer



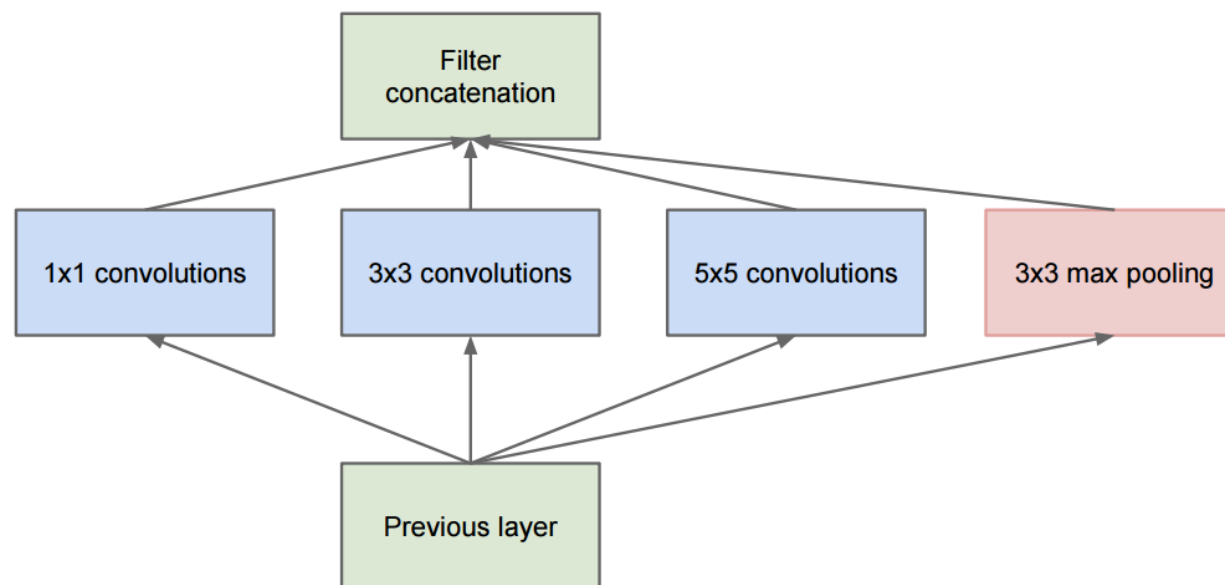
(b) Mlpconv layer



M. Lin, Q. Chen, and S. Yan, [Network in network](#), ICLR 2014



GoogLeNet: ILSVRC 2014 winner

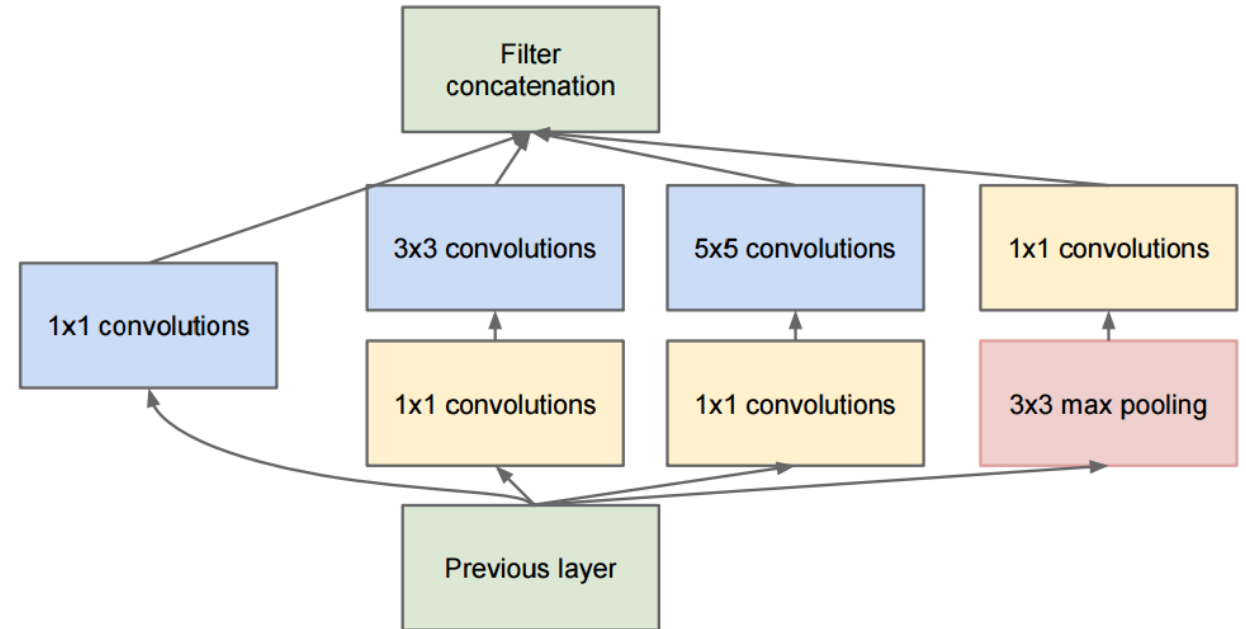


➤ The Inception Module:

- Parallel paths with different receptive field sizes and operations are meant to capture sparse patterns of correlations in the stack of feature maps



GoogLeNet: ILSVRC 2014 winner



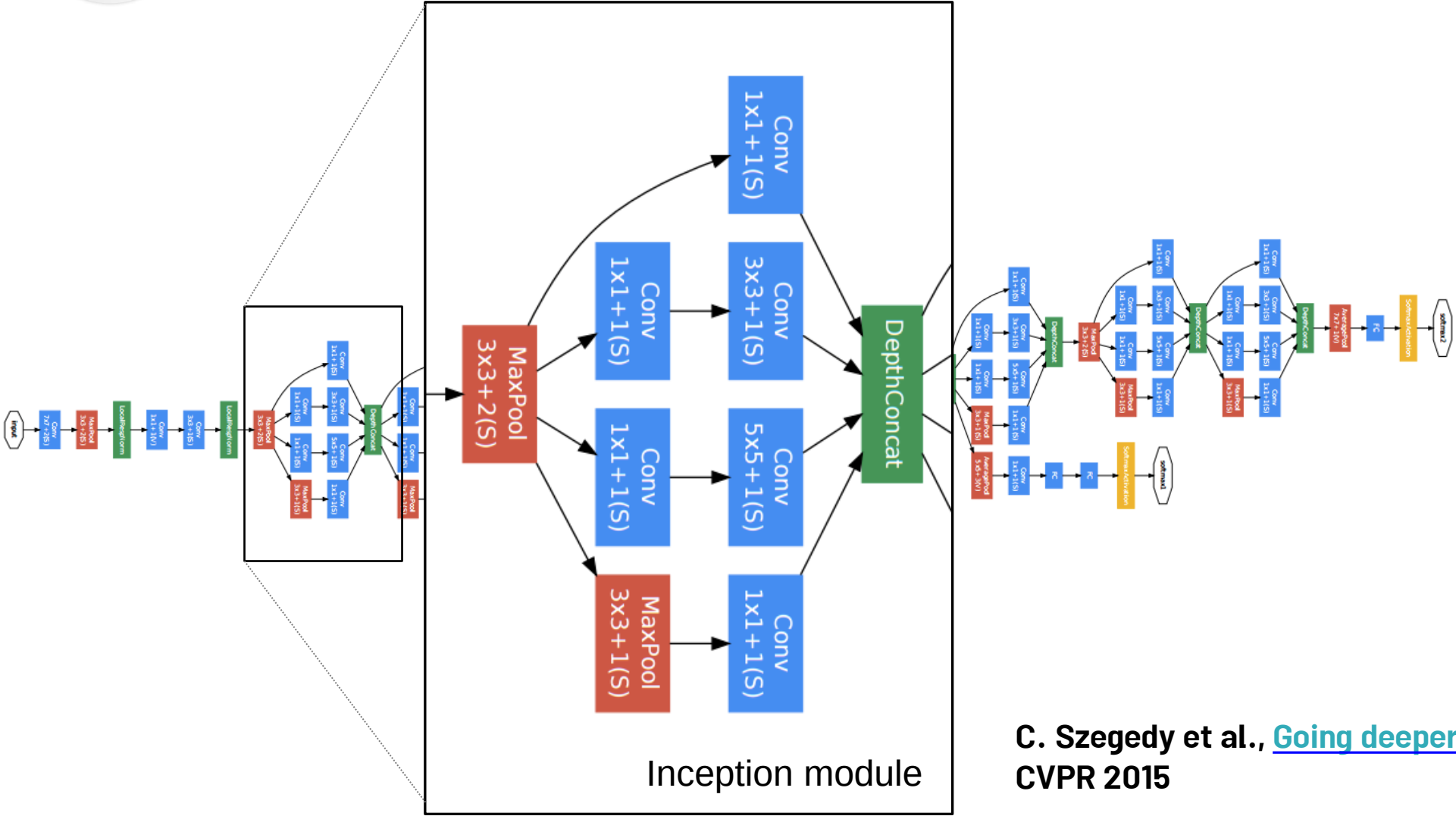
➤ The Inception Module:

- Parallel paths with different receptive field sizes and operations are meant to capture sparse patterns of correlations in the stack of feature maps
- Use 1x1 convolutions for dimensionality reduction before expensive convolutions

C. Szegedy et al., [Going deeper with convolutions](#), CVPR 2015



GoogLeNet: ILSVRC 2014 winner

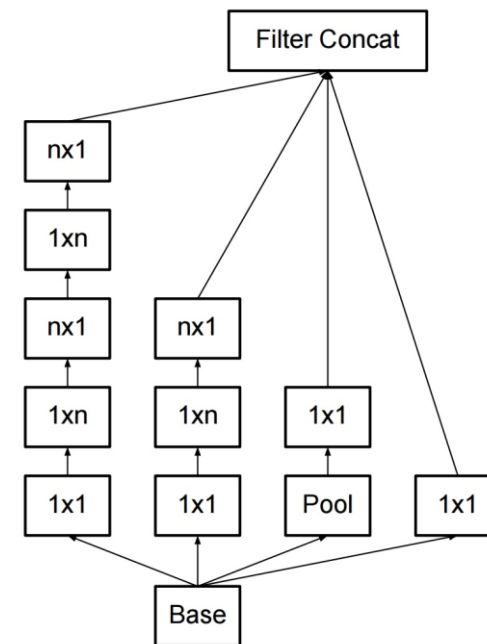
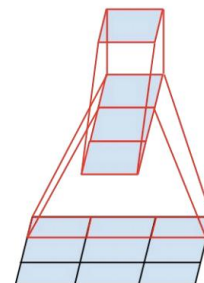
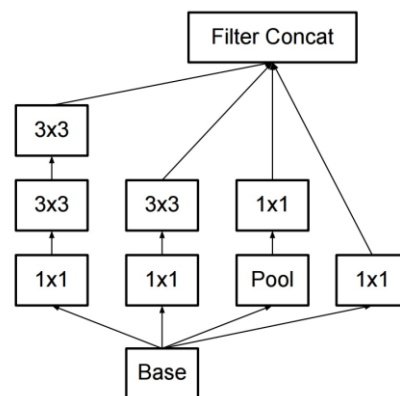
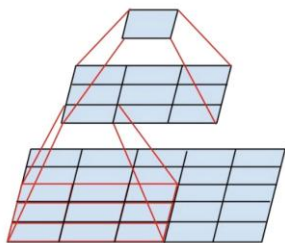


C. Szegedy et al., [Going deeper with convolutions](#), CVPR 2015





Inception v2, v3

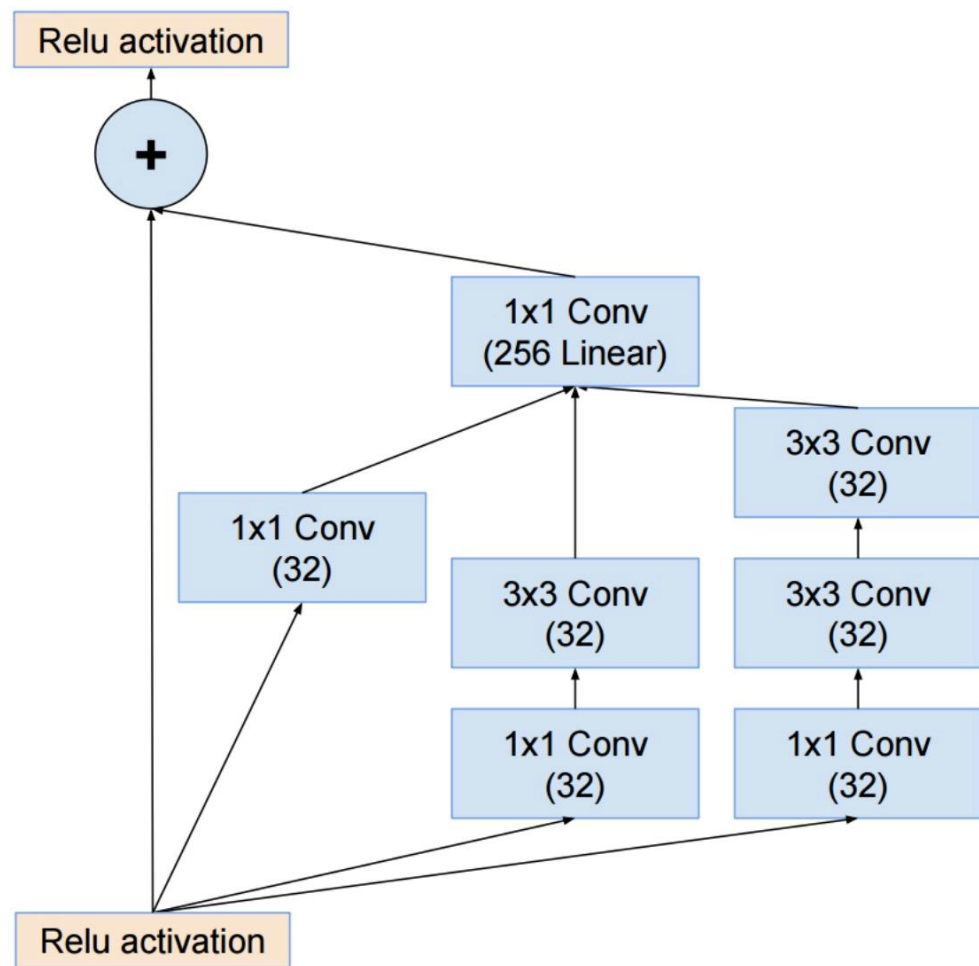


➤ Inception v2, v3:

- Regularize training with **batch normalization**, reducing importance of auxiliary classifiers
- More variants of inception modules with aggressive factorization of filters



Inception v4



C. Szegedy et al., [Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning](#), AAAI 2017



GoogLeNet: ILSVRC 2014 winner

Team	Year	Place	Error (top-5)	External data
SuperVision Toronto (AlexNet, layers)	2012	-	16.4%	no
SuperVision	2012	1st	15.3%	ImageNet 22k
Clarifai – NYU (7 layers)	2013	-	11.7%	no
Clarifai	2013	1st	11.2%	ImageNet 22k
VGG – Oxford (16 layers)	2014	2nd	7.32%	no
GoogLeNet (19 layers)	2014	1st	6.67%	no
ResNet (152 layers)	2015	1st	3.57%	
Human expert*			5.1%	

<http://karpathy.github.io/2014/09/02/what-i-learned-from-competing-against-a-convnet-on-imagenet/>



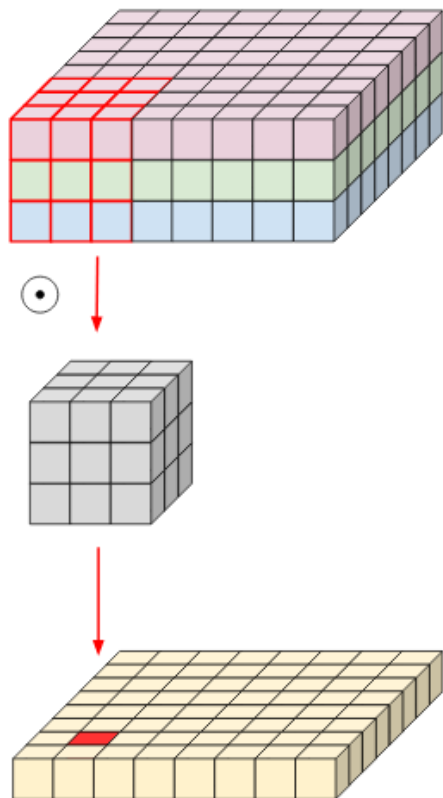
Design principles

- **Reduce filter sizes** (except possibly at the lowest layer), factorize filters aggressively
- **Use 1x1 convolutions** to reduce and expand the number of feature maps judiciously
- **Use skip connections** and/or create **multiple paths** through the network

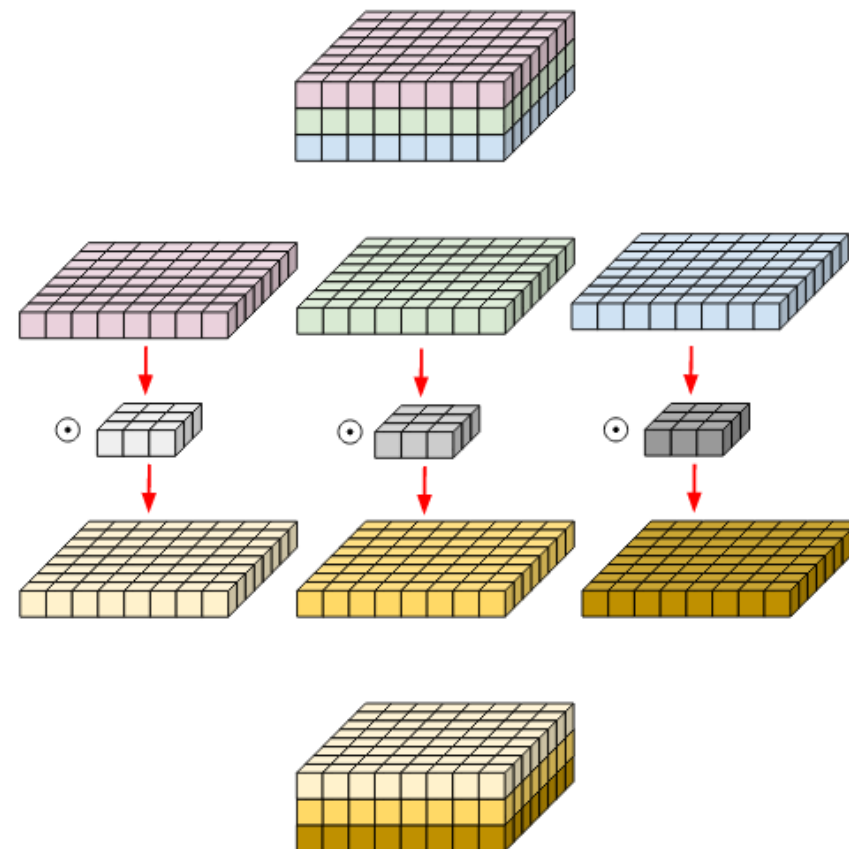
Depth-wise Convolution



Depth-wise Convolution



Normal Convolution



Depth-wise Convolution



Depth-wise separable Convolution

- It is originated from the idea that the convolution in depth and spatial dimensions **can be separated**
- For example consider sobel filter in image processing which is separable convolution in height and width dimension.

Gx filter can be viewed as matrix product of $[1 \ 2 \ 1]$ transpose with $[-1 \ 0 \ 1]$.

This separable structure reduces parameters from 9 to 6.

-1	0	+1
-2	0	+2
-1	0	+1

Gx

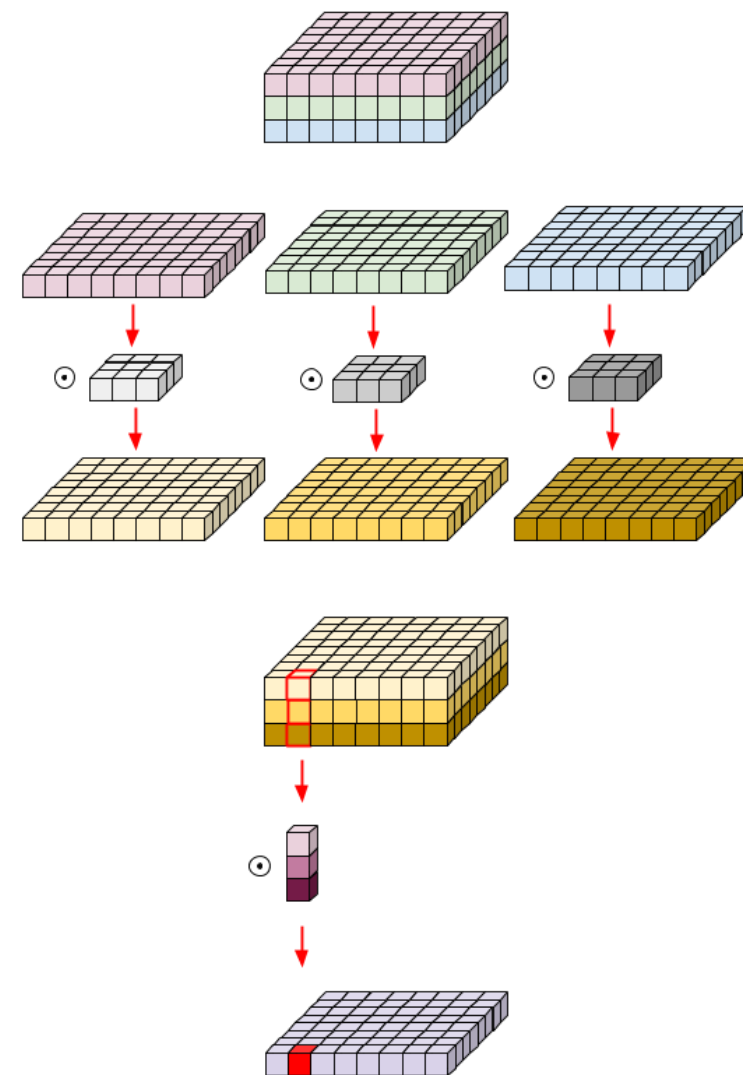
+1	+2	+1
0	0	0
-1	-2	-1

Gy



Depth-wise separable Convolution

- Depth-wise separable convolution:
separate depth dimension from horizontal (width*height)
- we perform depth-wise convolution and after that we use a 1×1 filter to cover the depth dimension
- Huge reduction in the number of parameters



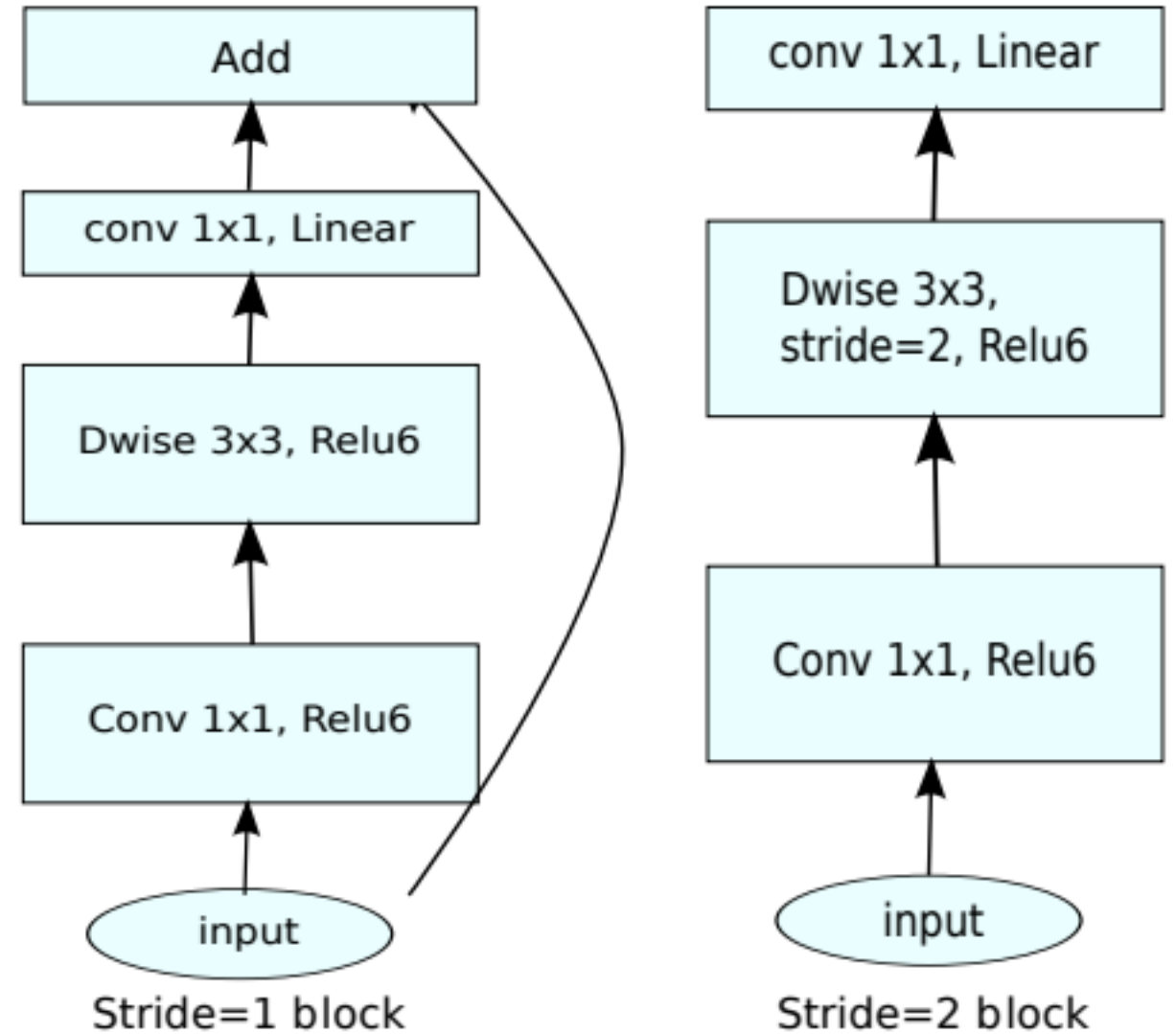
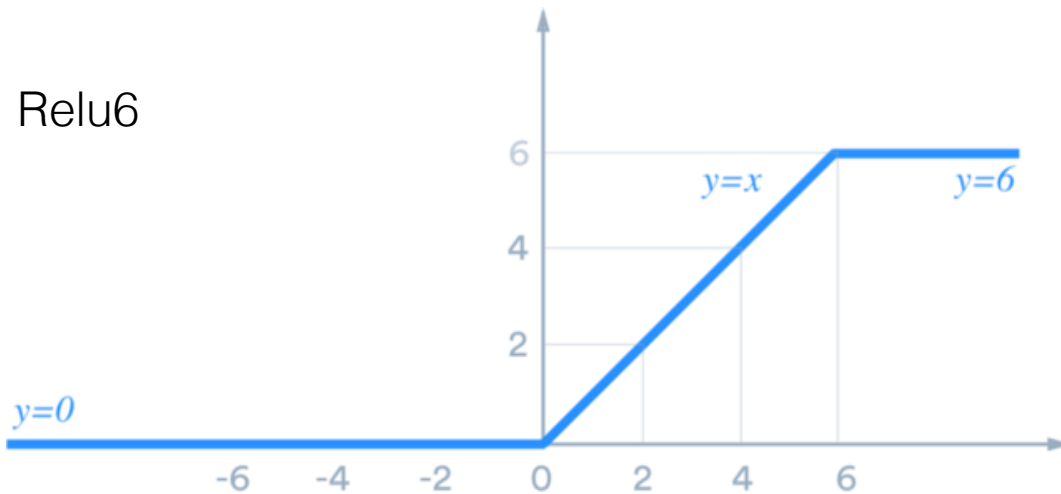


MobileNet V2

Input	Operator	Output
$h \times w \times k$	1x1 conv2d, ReLU6	$h \times w \times (tk)$
$h \times w \times tk$	3x3 dwse s=s, ReLU6	$\frac{h}{s} \times \frac{w}{s} \times (tk)$
$\frac{h}{s} \times \frac{w}{s} \times tk$	linear 1x1 conv2d	$\frac{h}{s} \times \frac{w}{s} \times k'$

Table 1: *Bottleneck residual block* transforming from k to k' channels, with stride s , and expansion factor t .

Relu6

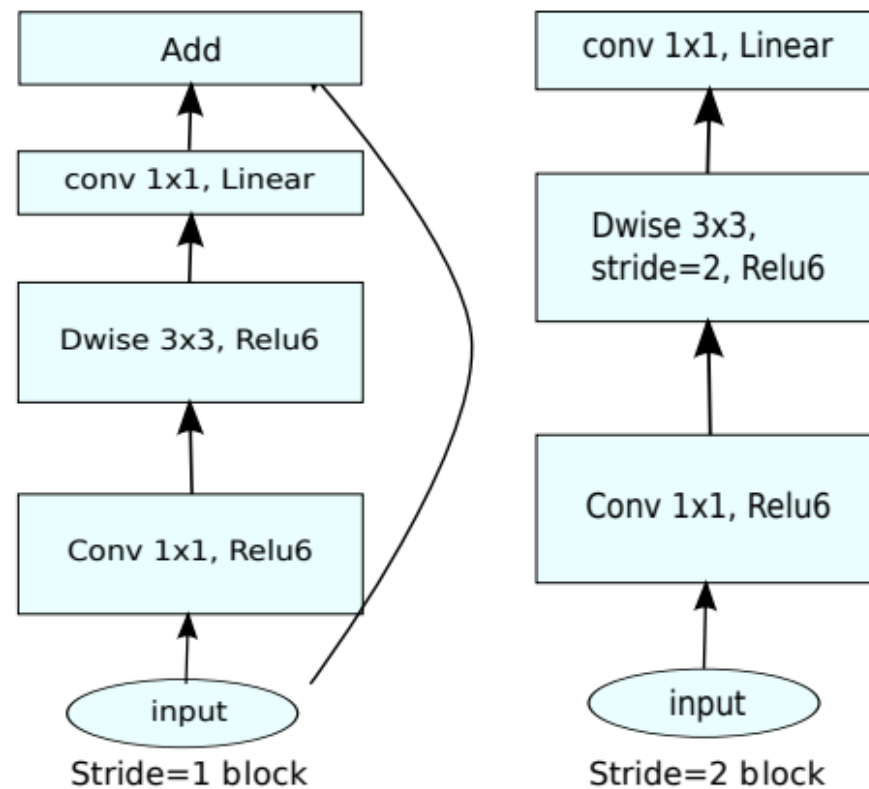




MobileNet V2

Input	Operator	t	c	n	s
$224^2 \times 3$	conv2d	-	32	1	2
$112^2 \times 32$	bottleneck	1	16	1	1
$112^2 \times 16$	bottleneck	6	24	2	2
$56^2 \times 24$	bottleneck	6	32	3	2
$28^2 \times 32$	bottleneck	6	64	4	2
$14^2 \times 64$	bottleneck	6	96	3	1
$14^2 \times 96$	bottleneck	6	160	3	2
$7^2 \times 160$	bottleneck	6	320	1	1
$7^2 \times 320$	conv2d 1x1	-	1280	1	1
$7^2 \times 1280$	avgpool 7x7	-	-	1	-
$1 \times 1 \times 1280$	conv2d 1x1	-	k	-	-

Table 2: MobileNetV2 : Each line describes a sequence of 1 or more identical (modulo stride) layers, repeated n times. All layers in the same sequence have the same number c of output channels. The first layer of each sequence has a stride s and all others use stride 1. All spatial convolutions use 3×3 kernels. The expansion factor t is always applied to the input size as described in Table 1.

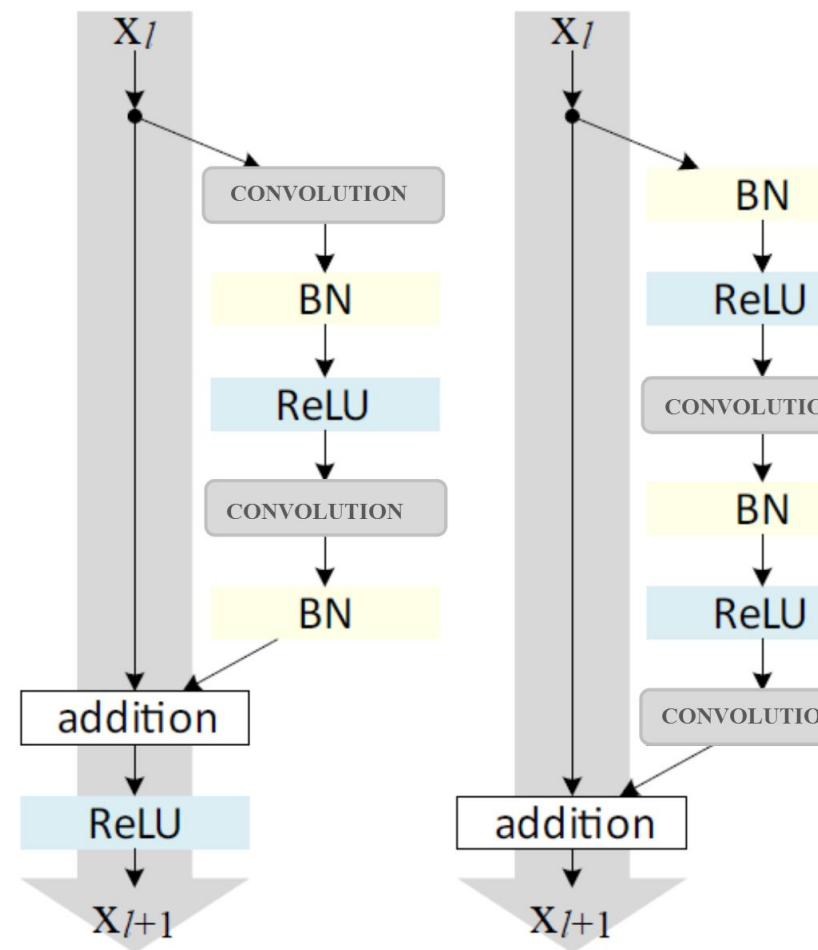


Squeeze and Excitation



Improving Resnets

- Creates a more direct path for propagating information throughout network (moves activation to residual mapping pathway)
- Gives better performance



Original Resnet Module

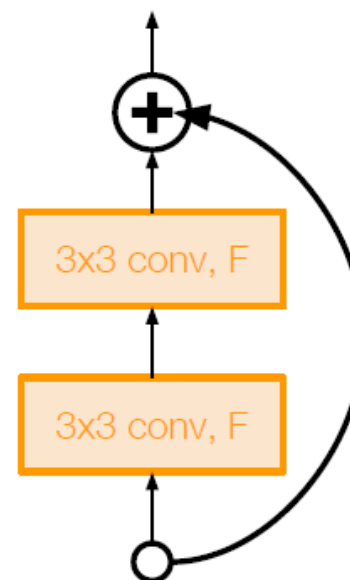
Identity Resnet Module

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, [Identity Mappings in Deep Residual Networks](#), ECCV 2016

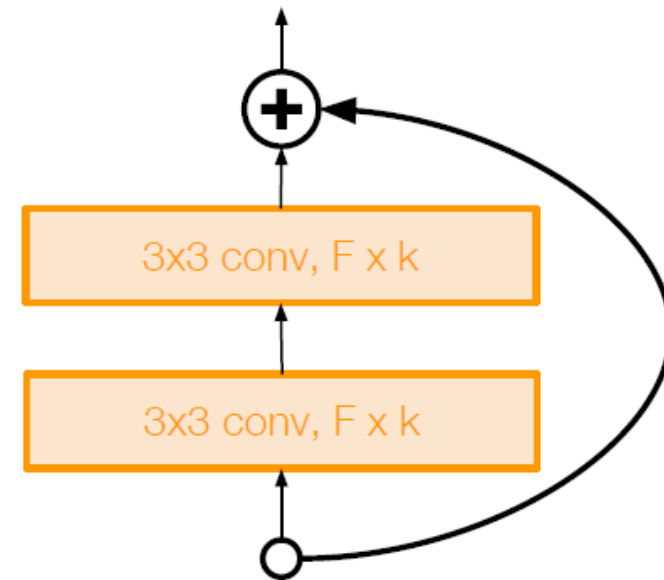


Improving Resnets

- Argues that residuals are the important factor, not depth
- Use wider residual blocks ($F \times k$ filters instead of F filters in each layer)
- 50-layer wide ResNet outperforms 152-layer original ResNet
- Increasing width instead of depth more computationally efficient (parallelizable)



Basic residual block



Wide residual block



ResNeXt

- Also from creators of ResNet
- Increases width of residual block through multiple parallel pathways ("cardinality")
- Parallel pathways similar in spirit to Inception module

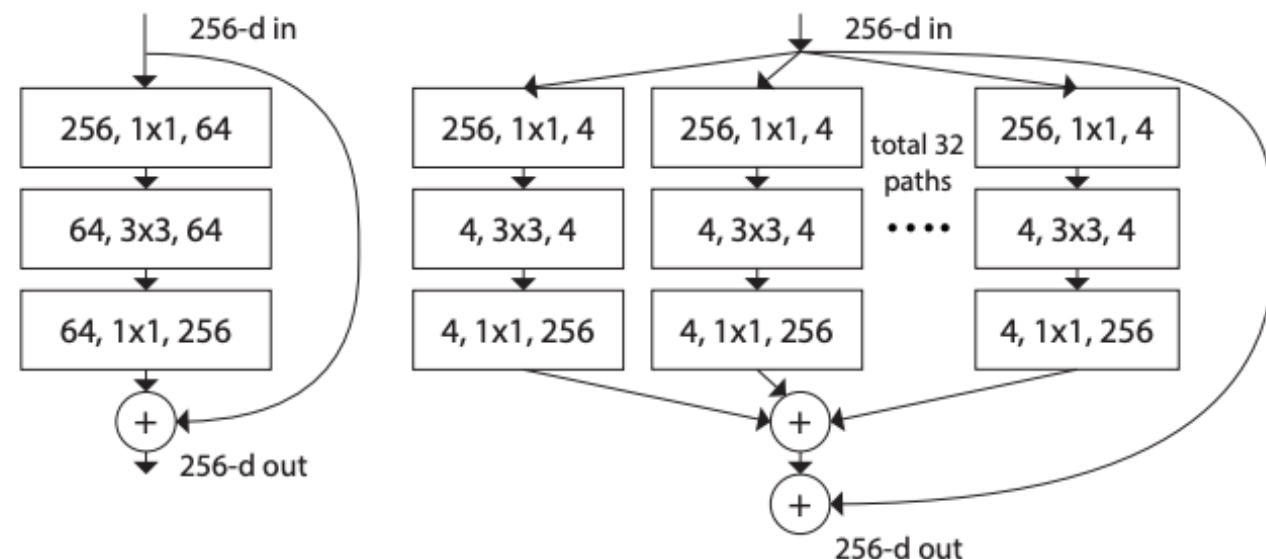


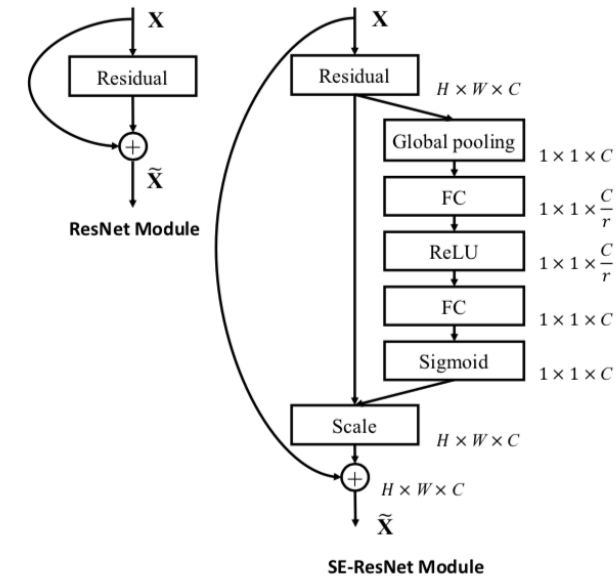
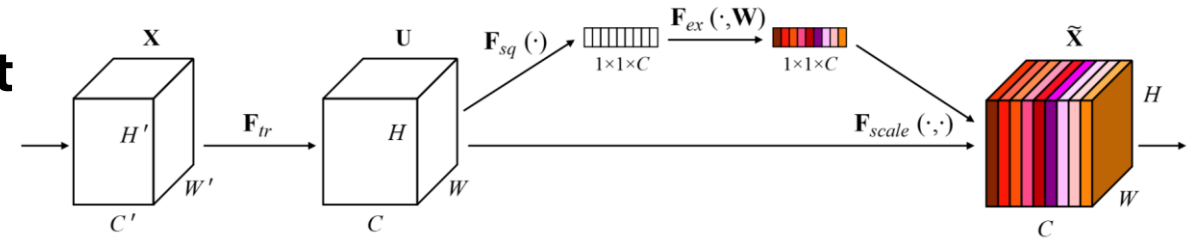
Figure 1. **Left:** A block of ResNet [14]. **Right:** A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He, [Aggregated residual transformations for deep neural networks](#), CVPR 2017



Squeeze-and-Excitation Networks (SENet)

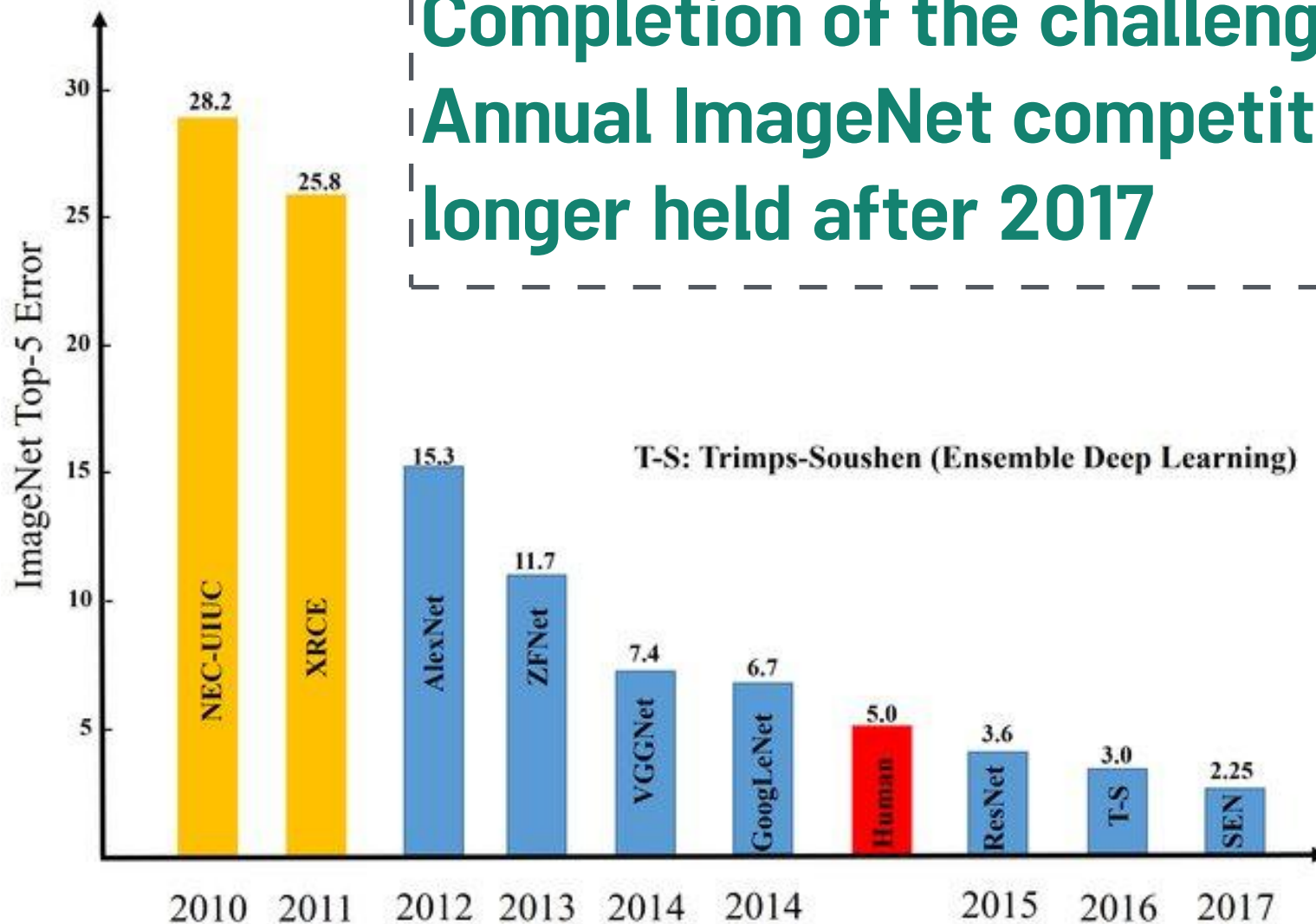
- Add a “feature recalibration” module that learns to adaptively reweight feature maps
- Global information (global avg. pooling layer) + 2 FC layers used to determine feature map weights
- ILSVRC'17 classification winner (using ResNeXt-152 as a base architecture)





Comparison

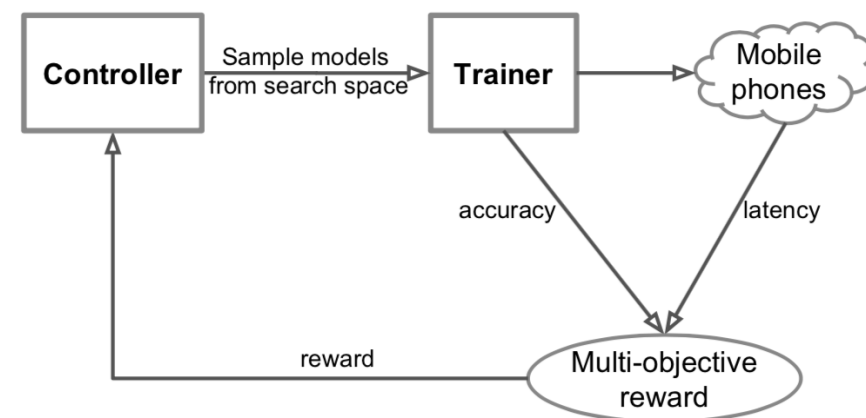
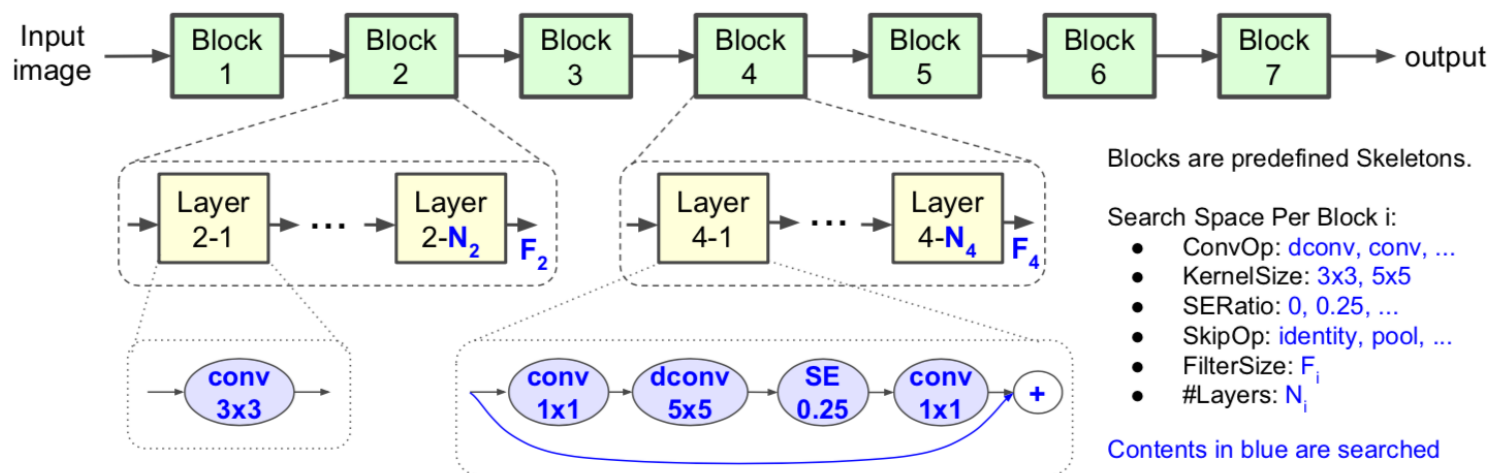
**Completion of the challenge:
Annual ImageNet competition no
longer held after 2017**



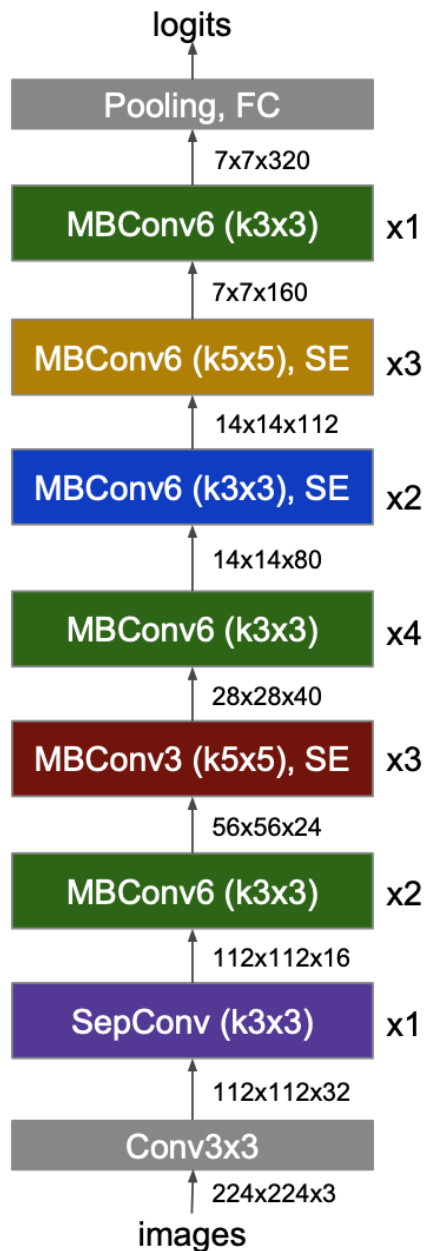
Neural Architecture Search



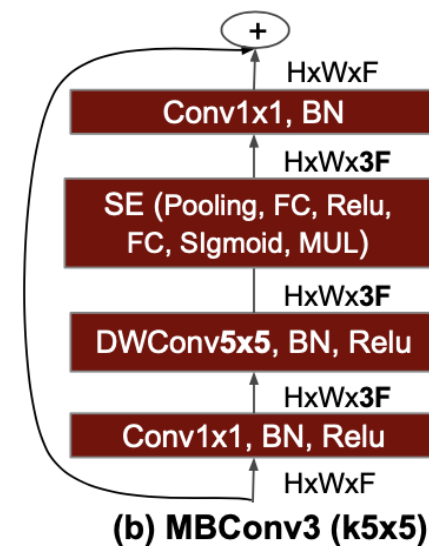
GoogLeNet: ILSVRC 2014 winner



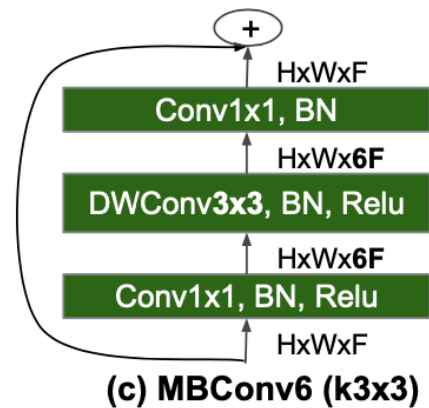
Tan, Mingxing, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le., [Mnasnet: Platform-aware neural architecture search for mobile](#), CVPR 2019



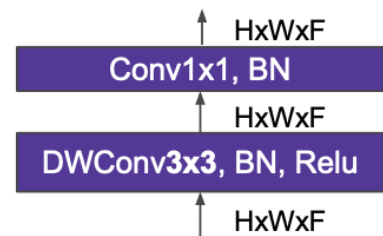
(a) MnasNet-A1



(b) MBConv3 (k5x5)



(c) MBConv6 (k3x3)



(d) SepConv (k3x3)



MnasNet

Model	Type	#Params	#Mult-Adds	Top-1 Acc. (%)	Top-5 Acc. (%)	Inference Latency
MobileNetV1 [11]	manual	4.2M	575M	70.6	89.5	113ms
SqueezeNext [5]	manual	3.2M	708M	67.5	88.2	-
ShuffleNet (1.5x) [33]	manual	3.4M	292M	71.5	-	-
ShuffleNet (2x)	manual	5.4M	524M	73.7	-	-
ShuffleNetV2 (1.5x) [24]	manual	-	299M	72.6	-	-
ShuffleNetV2 (2x)	manual	-	597M	75.4	-	-
CondenseNet (G=C=4) [14]	manual	2.9M	274M	71.0	90.0	-
CondenseNet (G=C=8)	manual	4.8M	529M	73.8	91.7	-
MobileNetV2 [29]	manual	3.4M	300M	72.0	91.0	75ms
MobileNetV2 (1.4x)	manual	6.9M	585M	74.7	92.5	143ms
NASNet-A [36]	auto	5.3M	564M	74.0	91.3	183ms
AmoebaNet-A [26]	auto	5.1M	555M	74.5	92.0	190ms
PNASNet [19]	auto	5.1M	588M	74.2	91.9	-
DARTS [21]	auto	4.9M	595M	73.1	91	-
MnasNet-A1	auto	3.9M	312M	75.2	92.5	78ms
MnasNet-A2	auto	4.8M	340M	75.6	92.7	84ms
MnasNet-A3	auto	5.2M	403M	76.7	93.3	103ms

Table 1: **Performance Results on ImageNet Classification** [28]. We compare our MnasNet models with both manually-designed mobile models and other automated approaches – *MnasNet-A1* is our baseline model; *MnasNet-A2* and *MnasNet-A3* are two models (for comparison) with different latency from the same architecture search experiment; *#Params*: number of trainable parameters; *#Mult-Adds*: number of multiply-add operations per image; *Top-1/5 Acc.*: the top-1 or top-5 accuracy on ImageNet validation set; *Inference Latency* is measured on the big CPU core of a Pixel 1 Phone with batch size 1.



EfficientNet

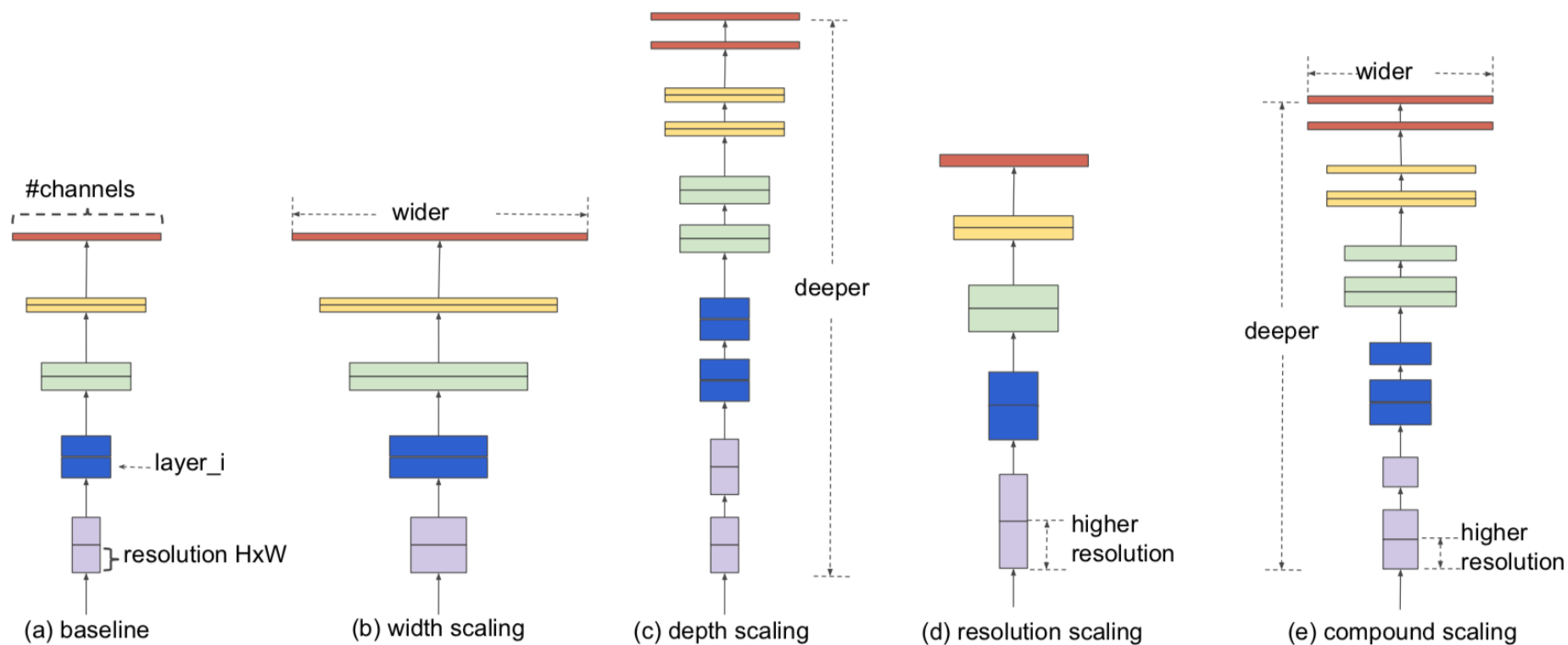
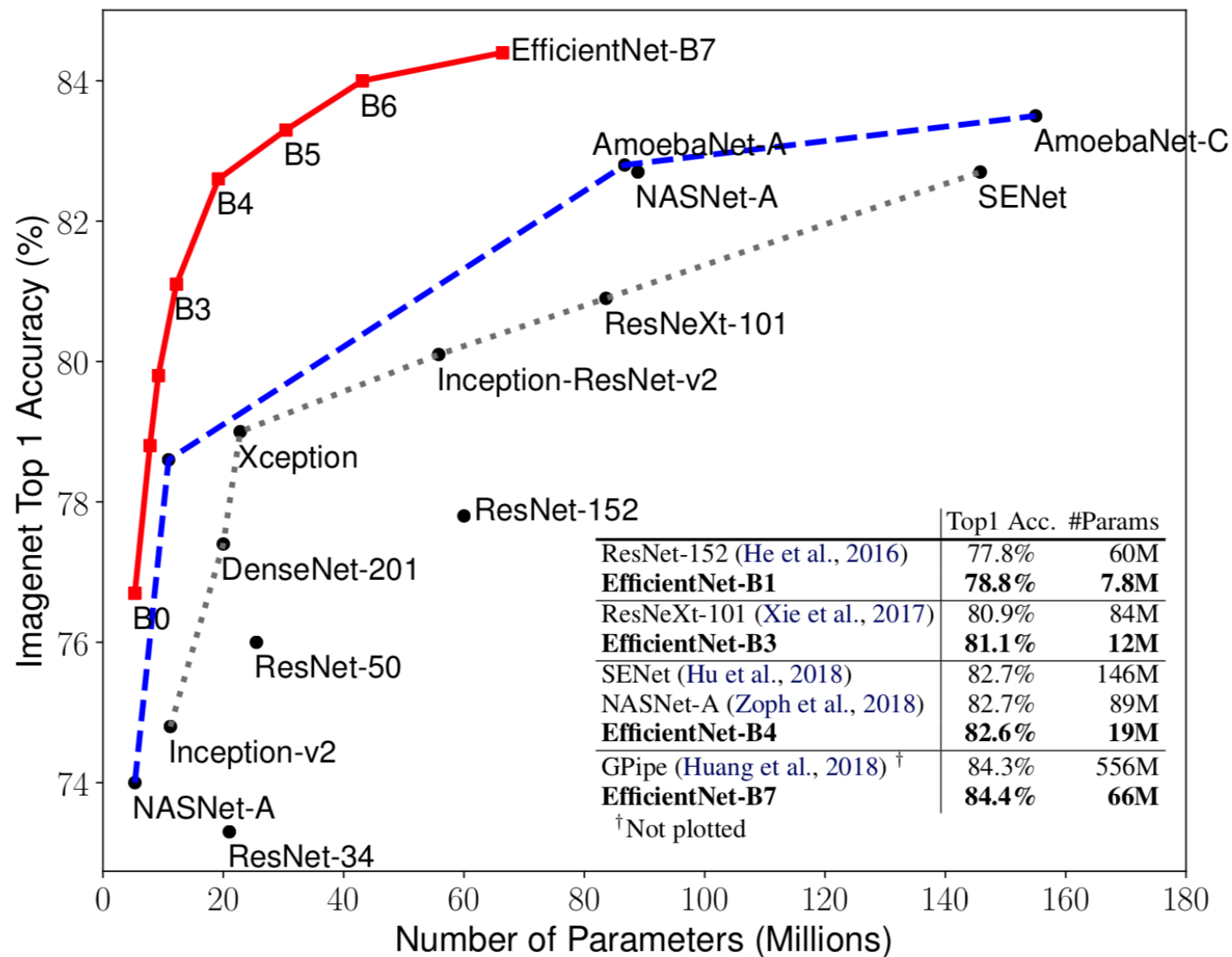


Figure 2. Model Scaling. (a) is a baseline network example; (b)-(d) are conventional scaling that only increases one dimension of network width, depth, or resolution. (e) is our proposed compound scaling method that uniformly scales all three dimensions with a fixed ratio.



EfficientNet



2 DNN

Convolutional NNs

- Basics of CNNs
- Modern CNNs
- **Batch Normalization**
- Transfer Learning



Batch Normalization

- **Internal Covariance Shift:**
 - **Distribution of inputs to a layer** is changing during training
 - **Harder to train: smaller learning rate, careful initialization**
 - **Easier if distribution of inputs stayed same**
 - **How to enforce same distribution?**



Batch Normalization

- Make **mean = 0** and standard **deviation = 1**
- Fast and can backprop through it
- How to **compute the statistics**?



Batch Normalization

- **How to compute the statistics**
 - **Going over the entire dataset** is too slow
 - **Idea: the batch is an approximation of the dataset**
 - **Compute statistics over the batch**



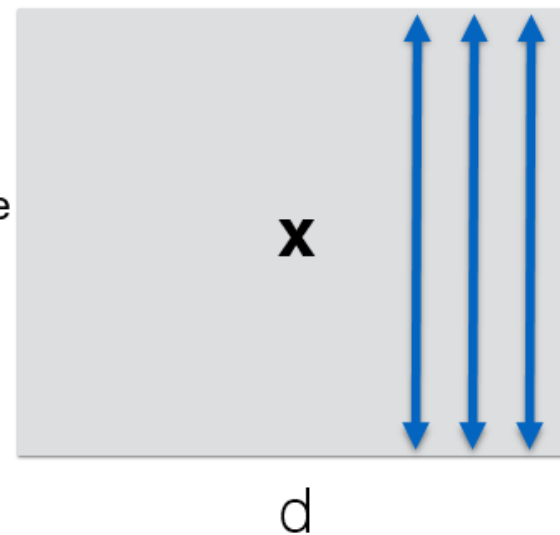
Batch Normalization

Compute the empirical mean and variance independently for each dimension

Mean $\mu_{\mathcal{B}}$ $\leftarrow \frac{1}{m} \sum_{i=1}^m x_i$

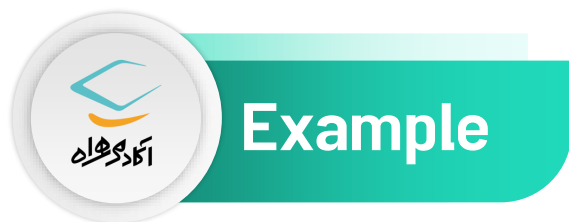
Batch size m

Variance $\sigma_{\mathcal{B}}^2$ $\leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$



Normalize

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$



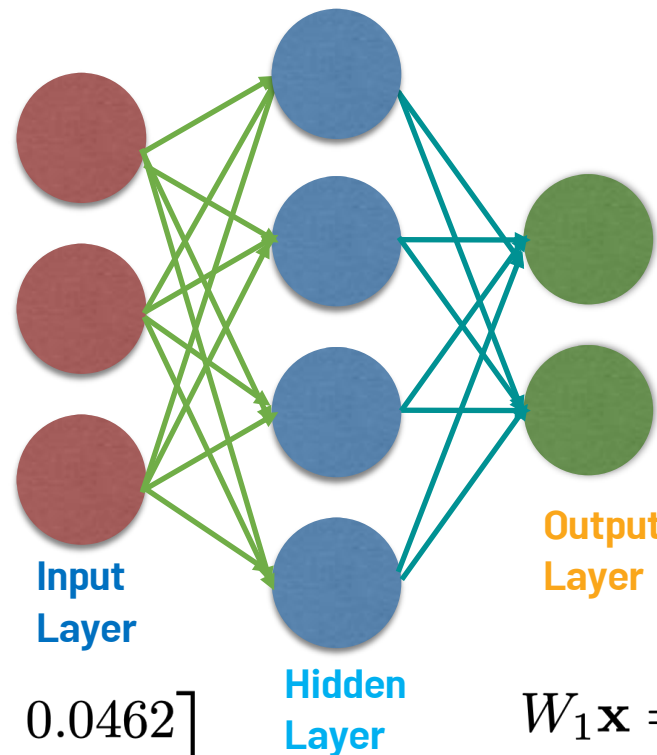
Example

Consider a batch of data

$$\mathbf{x} = \begin{bmatrix} 0.6787 & 0.3922 & 0.7060 & 0.0462 \\ 0.7577 & 0.6555 & 0.0318 & 0.0971 \\ 0.7431 & 0.1712 & 0.2769 & 0.8235 \end{bmatrix}$$

Batch Normalization

$$\mu_{\mathcal{B}} = \begin{bmatrix} -0.2068 \\ 0.7061 \\ 1.0671 \\ 1.1479 \end{bmatrix} \sigma_{\mathcal{B}} = \begin{bmatrix} 0.3634 \\ 0.6784 \\ 0.7042 \\ 0.4630 \end{bmatrix}$$



$$W_1 = \begin{bmatrix} -0.2503 & -0.7794 & 0.4126 \\ 0.1978 & 1.7522 & -0.1184 \\ 2.0660 & 0.4188 & -0.0716 \\ 1.5989 & -0.0364 & 0.8601 \end{bmatrix}$$

$$W_1 \mathbf{x} = \begin{bmatrix} -0.4538 & -0.5384 & -0.0873 & 0.2525 \\ 1.3740 & 1.2059 & 0.1627 & 0.0819 \\ 1.6664 & 1.0726 & 1.4522 & 0.0771 \\ 1.6968 & 0.7505 & 1.3659 & 0.7785 \end{bmatrix}$$

$$(W_1 \mathbf{x} - \mu_{\mathcal{B}}) / \sigma_{\mathcal{B}} = \begin{bmatrix} -0.6798 & -0.9126 & 0.3287 & 1.2637 \\ 0.9845 & 0.7366 & -0.8010 & -0.9201 \\ 0.8511 & 0.0079 & 0.5469 & -1.4059 \\ 1.1854 & -0.8584 & 0.4708 & -0.7978 \end{bmatrix}$$

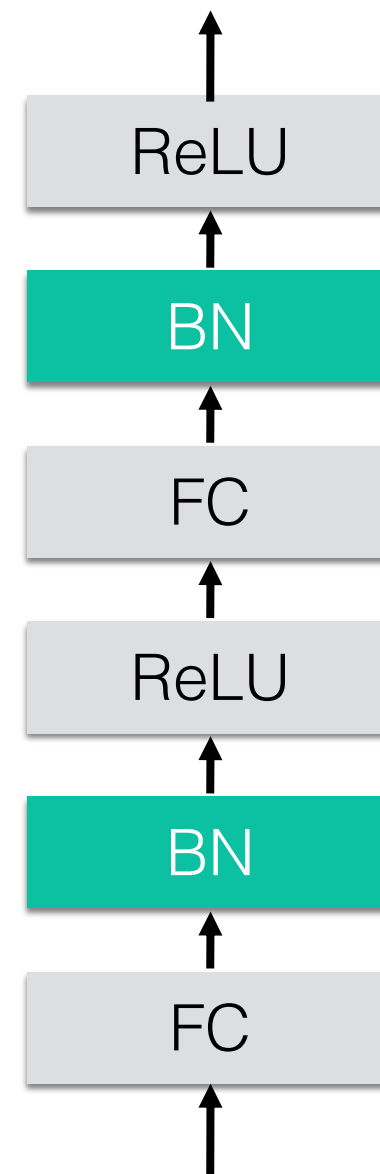


Batch Normalization

- Usually inserted after fully-connected or convolutional layers and before non-linearity

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

Not necessarily want Zero-mean unit-variance
Gaussian input before tanh



Sergey Ioffe, and Christian Szegedy., [Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift](#), ICML 2015



Batch Normalization

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.



Recap: Regularization

- A common pattern in regularization is like drop-out
- In training, we **add** some sort of randomness
- In testing, we **average out** randomness



Batch Normalization

- **Want deterministic inference**
- **Different test batches will give different results**
- **Solution: precompute mean and variance on training set and use for inference**
- **Practically: maintain running average of statistics during training**



Batch Normalization

Input: Network N with trainable parameters Θ ;
subset of activations $\{x^{(k)}\}_{k=1}^K$

Output: Batch-normalized network for inference, $N_{\text{BN}}^{\text{inf}}$

- 1: $N_{\text{BN}}^{\text{tr}} \leftarrow N$ // Training BN network
- 2: **for** $k = 1 \dots K$ **do**
- 3: Add transformation $y^{(k)} = \text{BN}_{\gamma^{(k)}, \beta^{(k)}}(x^{(k)})$ to $N_{\text{BN}}^{\text{tr}}$ (Alg. 1)
- 4: Modify each layer in $N_{\text{BN}}^{\text{tr}}$ with input $x^{(k)}$ to take $y^{(k)}$ instead
- 5: **end for**
- 6: Train $N_{\text{BN}}^{\text{tr}}$ to optimize the parameters $\Theta \cup \{\gamma^{(k)}, \beta^{(k)}\}_{k=1}^K$
- 7: $N_{\text{BN}}^{\text{inf}} \leftarrow N_{\text{BN}}^{\text{tr}}$ // Inference BN network with frozen parameters
- 8: **for** $k = 1 \dots K$ **do**
- 9: // For clarity, $x \equiv x^{(k)}$, $\gamma \equiv \gamma^{(k)}$, $\mu_{\mathcal{B}} \equiv \mu_{\mathcal{B}}^{(k)}$, etc.
- 10: Process multiple training mini-batches $\mathcal{B}$, each of size m , and average over them:

$$\mathbb{E}[x] \leftarrow \mathbb{E}_{\mathcal{B}}[\mu_{\mathcal{B}}]$$
$$\text{Var}[x] \leftarrow \frac{m}{m-1} \mathbb{E}_{\mathcal{B}}[\sigma_{\mathcal{B}}^2]$$
- 11: In $N_{\text{BN}}^{\text{inf}}$, replace the transform $y = \text{BN}_{\gamma, \beta}(x)$ with

$$y = \frac{\gamma}{\sqrt{\text{Var}[x] + \epsilon}} \cdot x + \left(\beta - \frac{\gamma \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \right)$$
- 12: **end for**

Moving Average

$$\mu := \mu + \alpha(\mu_{\mathcal{B}} - \mu)$$

$$\sigma := \sigma + \alpha(\sigma_{\mathcal{B}} - \sigma)$$

For getting unbiased estimate



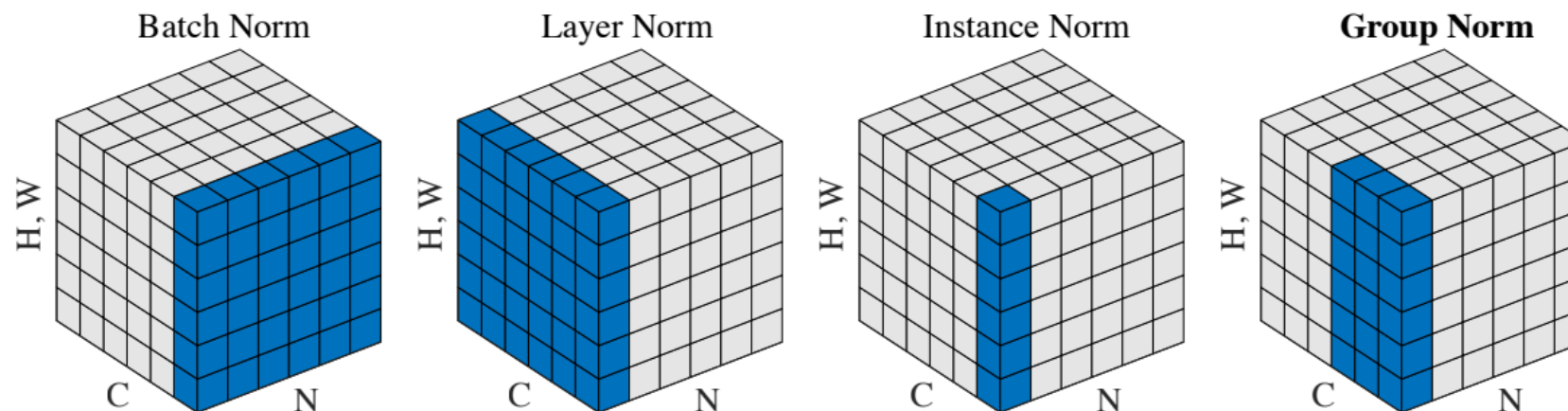
Batch Normalization

- **Advantages:**
 - Enables **higher learning rate** by stabilizing gradients
 - More resilient to parameter scale
 - **Regularizes model**
 - **Most CNN models use BN**



GoogLeNet: ILSVRC 2014 winner

- In BN, the statistics are computed across the batch and are the same for each example in the batch.
- In contrast, in LN, the statistics are computed across each feature and are independent of other examples.



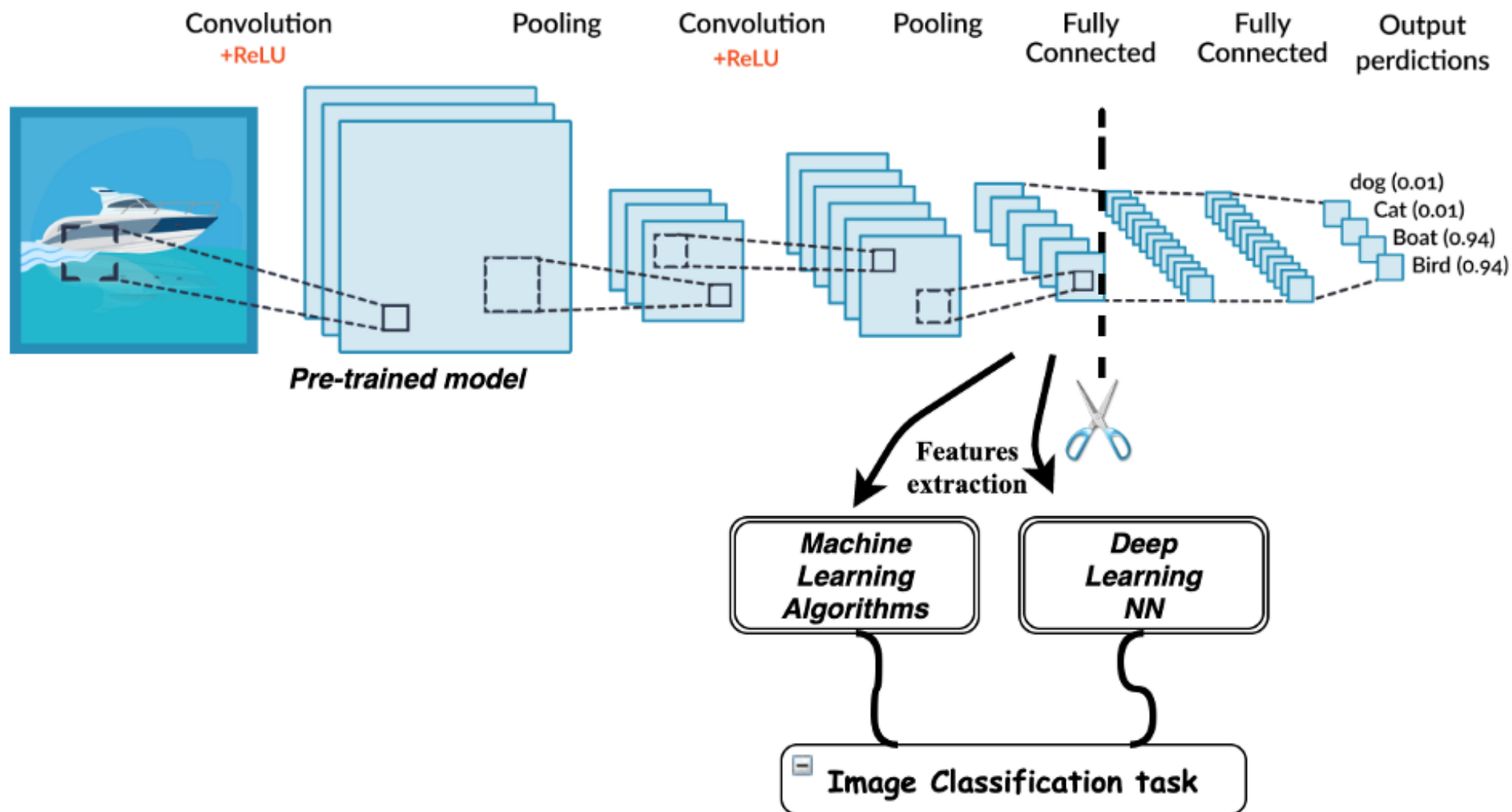
2 DNN

Convolutional NNs

- Basics of CNNs
- Modern CNNs
- Batch Normalization
- **Transfer Learning**

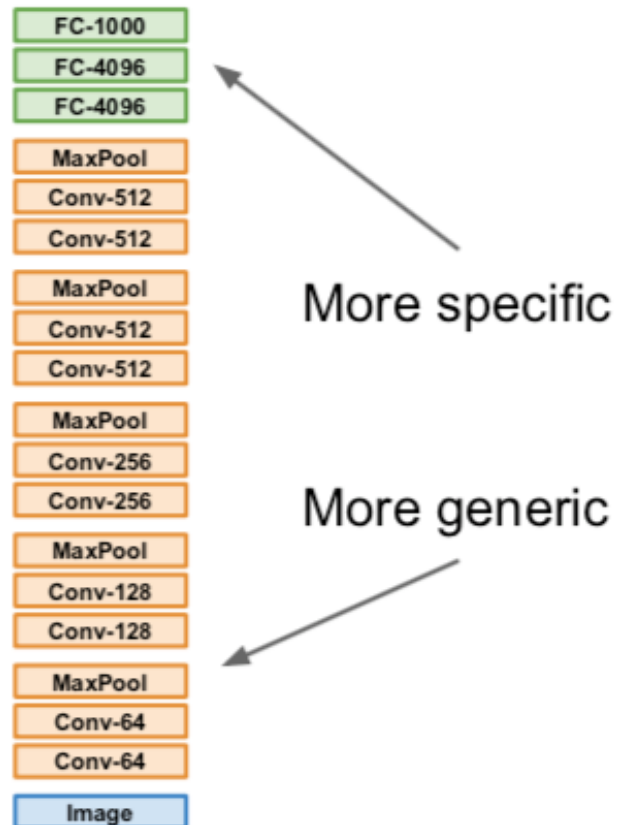


Transfer Learning





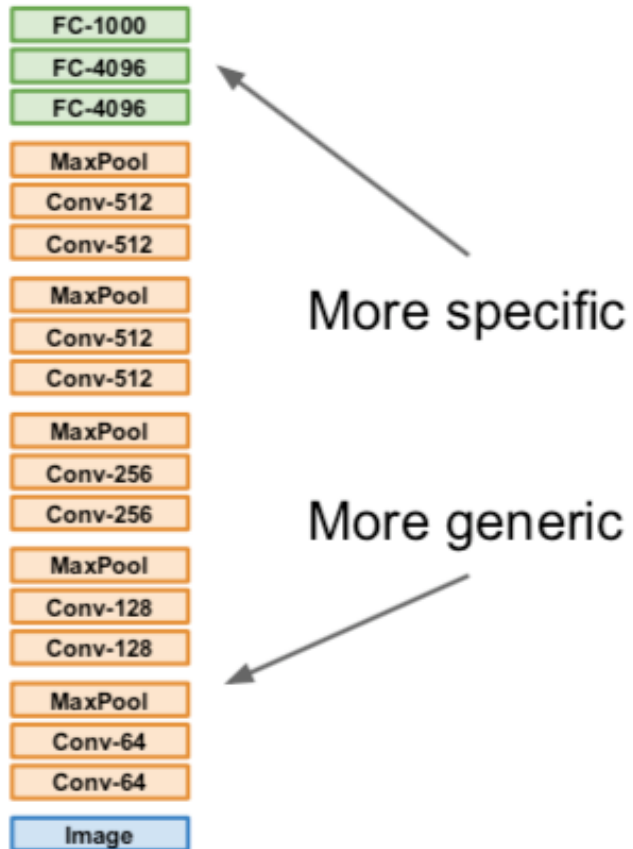
Transfer Learning



	very similar dataset	very different dataset
very little data	?	?
quite a lot of data	?	?



Transfer Learning



	very similar dataset	very different dataset
very little data	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data	Finetune a few layers	Finetune a larger number of layers



GoogLeNet: ILSVRC 2014 winner

➤ Topics:

- Convolution Neural Networks
- Batch Normalization

➤ Reference:

- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. Deep Learning. The MIT Press. **Chapters 9**
- Papers mentioned in the Slides